Modulo-3.pdf

michi_coder

Sistemas Operativos

2º Grado en Ingeniería del Software

Facultad de Informática
Universidad Complutense de Madrid

Accede al documento original



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

SHÔKO MADRID
EL EXAMEN SE APRUEBA ESTUDIANDO,
LA CARRERA SE VIVE SALIENDO.
Dale el último empujón a los apuntes y vente.
Pilla tu entrada
Nosotros ponemos la música y las copas para cuando termines el tema. Remata la faena y asegúrate la entrada ya, que vuelan.
Domingo
PURE
Sábado
THE ROOM
Viernes
Jueves
fuckin monday
Lunes

Módulo 3

3.1 Concepto de Proceso **2**
3.1.1 Creación y Finalización de Procesos 3
3.1.2 Modelo jerárquico 8
3.1.3 Estados de un Proceso 10
3.1.4 Estructuras de datos básicas para su gestión 12
3.2 Planificación **15**
3.2.1 Concepto de planificador 17
3.2.2 Estrategias de Planificación 19
3.3 Threads **22**
3.3.1 Concepto de Thread 24
3.3.2 Estructura de una Aplicación Multithread 26
3.3.3 Implementación de los Threads. Biblioteca POSIX Threads 28
3.4 Sincronización y Comunicación **30**
3.4.1 Concepto de Carrera y Definición de Sección Crítica 34
3.4.2 Exclusión mutua 35
3.4.3 Semáforos, cerrojos y variables condicionales 39
3.4.4 Soporte Hardware para Primitivas de Sincronización 45

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3.1 Concepto de Proceso

Concepto de Programa

Un programa es un archivo que reside en el disco duro. Este archivo contiene una serie de instrucciones escritas en un lenguaje de programación (como C) que le dicen al ordenador qué hacer.

En términos más sencillos: Un programa es como el icono de un juego o aplicación en tu escritorio. Es el archivo que contiene todas las instrucciones, pero no está "vivo" hasta que haces doble clic en él.

Ejemplo: Microsoft Word, Google Chrome, o cualquier juego son programas. Al abrirlos, se ejecutan, y es entonces cuando se convierten en procesos.

> Concepto Relacionado: Archivo Ejecutable
> Es un tipo especial de archivo que contiene un programa listo para ser ejecutado por el sistema operativo. En Windows, estos archivos suelen tener la extensión .exe. En Linux y macOS, no siempre tienen una extensión específica, pero tienen permisos de ejecución.

Concepto de Proceso

Un proceso es un programa en ejecución. Es decir, cuando haces doble clic en el icono de un programa (o lo ejecutas desde la línea de comandos), el sistema operativo crea un proceso. Este proceso usa recursos del sistema, como memoria y tiempo de CPU, para llevar a cabo las instrucciones del programa.

> Concepto Relacionado: Contexto de un Proceso
> Es toda la información que el sistema operativo necesita para gestionar ese proceso. Incluye el estado del procesador (los valores de los registros), qué partes de la memoria está usando el proceso, qué archivos tiene abiertos, etc. El sistema operativo usa esta información para poder detener temporalmente un proceso y luego reanudarlo más tarde, sin que el proceso "se dé cuenta".

Diferencias Clave entre Programa y Proceso

- Programa:
  - Es un archivo **pasivo** en el disco.
  - No consume recursos del sistema mientras no se esté ejecutando.
  - Es como una receta escrita.
- Proceso:
  - Es un programa **activo**, en ejecución.
  - Consume recursos del sistema (CPU, memoria, etc.).
  - Es como cocinar siguiendo la receta.
  - Puede haber **varios procesos** ejecutando el **mismo programa** al mismo tiempo.

Componentes de un Proceso

Cuando un **programa se convierte en proceso**, el sistema operativo organiza su memoria en varias secciones:

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

- **Código (o Texto)**: Aquí están las instrucciones del programa que la CPU debe ejecutar. Esta sección suele ser de solo lectura.
- **Datos**: Aquí se almacenan las variables globales y estáticas del programa.
- **Pila (Stack)**: La pila es una zona de memoria que se usa para:
    - Almacenar variables locales de las funciones.
    - Guardar la dirección de retorno de las funciones.
    - Pasar argumentos a las funciones.

  La pila crece y decrece a medida que se llaman y retornan funciones.
- **Montículo (Heap)**: El montículo es una zona de memoria que se usa para la **memoria dinámica**. Es decir, memoria que se reserva y se libera **durante la ejecución** del programa, usando funciones como malloc y free

Concepto Relacionado: Memoria Virtual (Módulo 5)
Cada proceso tiene su propio espacio de memoria virtual. Esto significa que cada proceso "cree" que tiene acceso a toda la memoria del ordenador, aunque en realidad no sea así. El sistema operativo se encarga de mapear la memoria virtual de cada proceso a la memoria física (RAM) real, y de usar el disco duro como espacio adicional si es necesario (esto se llama "swapping" o "paginación"). Gracias a la memoria virtual, los procesos están aislados unos de otros

3.1.1 Creación y Finalización de Procesos

La Familia exec

En sistemas Unix, las funciones de la familia exec son herramientas poderosas para ejecutar programas. Su función principal es **cargar un nuevo programa en la memoria del proceso actual y comenzar su ejecución**. Esencialmente, **reemplazan** el código, los datos, la pila y el montículo del proceso actual con los del nuevo programa.
Existen varias funciones en la familia exec, cada una con ligeras variaciones en cómo se pasan los argumentos y cómo se busca el ejecutable:
execl, execlp, execle, execv, execvp, execvpe
Aunque todas ejecutan un programa, se diferencian en dos aspectos principales:

1. Cómo se pasan los argumentos al nuevo programa (lista o vector).
2. Cómo se busca el ejecutable (ruta completa o búsqueda en PATH).

Diferencias entre las Funciones exec

Las letras al final del nombre de cada función indican sus características:

l (**list**): Estas funciones (execl, execlp, execle) reciben los argumentos del programa a ejecutar como una **lista** de argumentos, terminada por un puntero NULL. Cada argumento es una cadena de caracteres separada.

```
execl("/bin/ls", "ls", "-l", "-a", (char *)NULL); // Ejecuta "ls -l -a"
```

- "/bin/ls": Ruta completa al ejecutable.
- "ls": El nombre del programa (se pasa como primer argumento, ‘argv[0]’).
- "-l": Primer argumento.
- "-a": Segundo argumento.
- (char *)NULL: Obligatorio. Indica el final de la lista de argumentos.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

knoweats!
Estudiar, dormir y comer bien
Elige las tres. Con Knoweats lo tienes fácil: comida casera lista para que te centres en lo demás.
Comer bien



20€ DE DESCUENTO
con el código
20ESTUDIO

v (**vector**): Estas funciones (execv, execvp, execvpe) reciben los argumentos como un **array de punteros a caracteres** (un "vector" de cadenas). El último elemento del **array** debe ser un puntero NULL.

```
char *args[] = {"ls", "-l", "-a", NULL};
execv("/bin/ls", args); // Ejecuta "ls -l -a"
```

- "/bin/ls": Ruta completa al ejecutable.
- args: Un array de punteros a caracteres. Cada elemento del array es un argumento. El último elemento debe ser NULL.

p (path): Estas funciones (execlp, execvp, execvpe) buscan el programa a ejecutar en los directorios especificados en la variable de entorno PATH. Esto es muy conveniente, ya que no necesitas saber la ruta completa al ejecutable.

```
execlp("ls", "ls", "-l", "-a", (char *)NULL); // Busca "ls" en el PATH
```

- "ls": El nombre del programa a ejecutar. `execlp` lo buscará en el PATH. El resto de argumentos son como en execl.

e (environment): Estas funciones (execle, execvpe) permiten especificar un **entorno** personalizado para el nuevo programa (un conjunto de variables de entorno).

Ejemplos:

```
int main() {
    // Ejemplo con execlp (búsqueda en PATH y lista de argumentos)
    printf("Ejecutando ls -l con execlp...\n");
    execlp("ls", "ls", "-l", (char *)NULL);

    // Si llegamos aquí, es que execlp falló
    perror("execlp failed"); //Imprime un mensaje de error
    exit(EXIT_FAILURE); //Sale del programa
}

int main() {
    // Ejemplo con execv (ruta completa y vector de argumentos)
    char *args[] = {"/bin/ls", "-l", "-a", NULL}; // Array de argumentos
    printf("Ejecutando ls -l -a con execv...\n");
    execv("/bin/ls", args);

    // Si llegamos aquí, es que execv falló
    perror("execv failed");
    exit(EXIT_FAILURE);
}
```

¡Importante!
(exec no retorna si tiene éxito): Si una función de la familia exec tiene éxito, la imagen del proceso actual (código, datos, pila, etc.) se reemplaza por completo con la del nuevo

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

programa. Esto significa que el código que sigue a la llamada a exec nunca se ejecutará. Es como si el proceso original "desapareciera" y se convirtiera en el nuevo programa.

Por el contrario, si falla, devuelve -1. Es muy importante verificar, después de la llamada a una función exec, que esta haya tenido éxito, ya que, de lo contrario, se seguiría ejecutando el código del programa original.

```
int main() {
  printf("Este mensaje SI se imprime\n");
  execlp("ls", "ls", "-l", (char*) NULL);
  printf("Este mensaje NO se imprime (si execlp tiene éxito)\n"); //Esta línea NUNCA se ejecuta.
  perror("Fallo en execlp"); //Solo se ejecuta si execlp falla.
  exit(EXIT_FAILURE);
}
```

La Variable de Entorno PATH

La variable de entorno PATH es una pieza clave para que el sistema operativo sepa dónde buscar los programas ejecutables. Es una lista de directorios, separados por dos puntos

(':'). Cuando escribes un comando en la terminal (como 'ls', 'gcc', 'python', etc.), el sistema operativo busca el ejecutable correspondiente en los directorios listados en PATH, en el orden en que aparecen.

Ver el Contenido de PATH:

echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

Esto significa que, si escribes un comando como `ls`, el sistema operativo buscará el ejecutable `ls` en los siguientes directorios, en este orden:

1. /usr/local/sbin
2. /usr/local/bin
3. /usr/sbin
4. /usr/bin
5. /sbin
6. /bin

Si encuentra un ejecutable llamado 'ls' en alguno de estos directorios, lo ejecutará. Si no lo encuentra en ninguno de ellos, mostrará un **mensaje de error** ("command not found").

¿Por qué es importante el orden de los directorios en PATH?

Imagina que tienes **dos** versiones diferentes del programa `ls` instaladas en tu sistema:

- Una versión antigua en /usr/bin/ls.
- Una versión nueva en /usr/local/bin/ls.

Si tu PATH es como el ejemplo anterior (/usr/local/bin antes que /usr/bin), se ejecutará la versión nueva de `ls`. Si el orden fuera al revés (/usr/bin antes que /usr/local/bin), se ejecutará la versión antigua.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.





TECNOLOGÍA a PRECIO de ESTUDIANTE

VERANO como ESTRELLA DEL ROCK



webuy.com

CLIC AQUÍ

Modificar temporalmente el PATH (para la sesión actual):

```
export PATH=/mi/directorio:$PATH  # Añade /mi/directorio al *principio* de PATH
export PATH=$PATH:/mi/directorio  # Añade /mi/directorio al *final* de PATH
```

Creación de Procesos (fork)

Las funciones de la familia exec reemplazan la imagen del proceso actual. Pero, ¿qué pasa si quieres ejecutar un programa sin reemplazar el proceso actual? Para eso necesitas crear un nuevo proceso. En sistemas Unix, la forma más común de crear un nuevo proceso es usando la función fork().

¿Qué hace fork()?:

- fork() crea un nuevo proceso, llamado *proceso hijo*, que es una copia del proceso original (llamado *proceso padre*).
- Después de la llamada a fork(), ambos procesos (padre e hijo) continúan ejecutándose desde el mismo punto (la línea siguiente a la llamada a fork()).
- La única diferencia entre el proceso padre y el hijo es el valor de retorno de fork():
    - En el proceso hijo, fork() devuelve 0.
    - En el proceso padre, fork() devuelve el identificador de proceso (PID) del hijo.
    - Si fork() falla (por ejemplo, por falta de recursos), devuelve -1.

Ejemplo:

```
int main() {
    pid_t pid;

    printf("Antes de fork(). PID = %d\n", getpid()); // getpid() devuelve el PID del proceso actual

    pid = fork(); // Crear un nuevo proceso

    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // Código del proceso hijo
        printf("Soy el hijo. PID = %d, PPID = %d\n", getpid(), getppid()); // getppid() devuelve el PID
del padre
        execlp("ls", "ls", "-l", NULL); // Reemplazar la imagen del hijo con "ls -l"
        perror("execlp failed"); // Si execlp falla
        exit(EXIT_FAILURE);

    } else {
        // Código del proceso padre
        printf("Soy el padre. PID = %d, PID del hijo = %d\n", getpid(), pid);
        wait(NULL); // Esperar a que el hijo termine
        printf("El hijo ha terminado.\n");
    }

    printf("Esta línea la ejecutan tanto el padre como el hijo (si execlp falla).\n");
    return 0;
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

CARPE DIEM
Organiza tu futuro:
estudia hoy para destacar mañana.
En Carpe Diem te esperan cursos adaptados a ti.
Una forma fácil y real de avanzar profesionalmente.
¡Quiero formarme!
Acreditados
Económicos
Desde casa
Elige tu curso



Explicación del Ejemplo:

1. printf("Antes de fork()...: Esta línea la ejecuta solo el proceso padre antes de crear el hijo.
2. pid = fork();: Aquí se crea el nuevo proceso.
    - Si fork() falla, devuelve -1, y se muestra un mensaje de error.
    - Si fork() tiene éxito:
        - En el proceso hijo, `pid` será 0.
        - En el proceso padre, `pid` será el PID del hijo.
3. if (pid == -1) { ... }: Manejo de error de `fork()`.
4. else if (pid == 0) { ... }: Este es el código que ejecutará el *hijo*.
    - printf("Soy el hijo...");: El hijo imprime su PID y el PID de su padre (usando getppid()).
    - execlp("ls", "ls", "-l", NULL);: El hijo reemplaza su propia imagen con la del comando `ls -l`. A partir de aquí, el hijo ya no ejecuta el código de este programa, sino el código de `ls`.
    - perror("execlp failed"); y exit(EXIT_FAILURE);: Estas líneas solo se ejecutarán si execlp falla.
5. else { ... }: Este es el código que ejecutará el padre.
    - printf("Soy el padre...");: El padre imprime su PID y el PID del hijo.
    - wait(NULL);: El padre espera a que el hijo termine. La función wait() suspende la ejecución del padre hasta que uno de sus hijos termine. El argumento `NULL` indica que no nos interesa el código de salida del hijo.
    - printf("El hijo ha terminado.\n");: Esta línea se ejecuta después de que el hijo haya terminado.
6. printf("Esta línea la ejecutan ...");: Esta línea solo se ejecutará si `execlp` falla en el hijo. Si `execlp` tiene éxito, el código del hijo se reemplaza por el de `ls`, y esta línea nunca se ejecutará en el hijo.

Resultado de la ejecución: La salida de este programa será algo como:

```
Antes de fork(). PID = 1234
Soy el padre. PID = 1234, PID del hijo = 1235
Soy el hijo. PID = 1235, PPID = 1234
(Salida de ls -l)
El hijo ha terminado.
```

El orden de las líneas "Soy el padre..." y "Soy el hijo..." podría variar, ya que los dos procesos se ejecutan concurrentemente. Pero la salida de `ls -l` siempre estará entre esas dos líneas, porque el padre espera a que el hijo termine.

fork() crea un nuevo proceso, y exec() (en cualquiera de sus variantes) reemplaza la imagen del proceso actual con un nuevo programa. Juntos, son **lo más importante** para la **ejecución de programas en sistemas Unix**.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3.1.2 Modelo jerárquico

Modelo Jerárquico de Procesos en UNIX/Linux

¿Cómo se organizan los procesos?
En sistemas operativos como UNIX y Linux, los procesos no están sueltos, sino que se organizan en una **jerarquía**, como un árbol genealógico. Esta estructura jerárquica nos sirve para entender cómo funciona el sistema operativo y cómo interactúan los procesos entre sí.

El Proceso Raíz: init o systemd
En la cima de este árbol genealógico hay un proceso especial, el "padre" o "abuelo" de todos los demás. Tradicionalmente, este proceso se llamaba init, y tenía el *identificador de proceso (PID) 1*. En muchos sistemas modernos, init ha sido reemplazado por systemd, pero la idea básica es la misma: es el primer proceso que arranca el sistema operativo y es el ancestro de todos los demás procesos.

Analogía: Imagina un árbol genealógico. init/systemd sería el antepasado más antiguo, el que está en la raíz del árbol. Todos los demás procesos son sus descendientes.

Padres e Hijos
Cuando un proceso crea un nuevo proceso (usando la función fork()), decimos que el proceso original es el *proceso padre* y el nuevo proceso es el *proceso hijo*. El hijo hereda:

- *Variables de entorno* (incluida la importante variable PATH).
- *Descriptores de archivo abiertos*.
- *Directorio de trabajo actual*.

Esta herencia es muy útil (tanto para las prácticas como para la sincronía), porque permite que el proceso padre configure un entorno para el hijo antes de que este empiece a ejecutarse.

El Árbol de Procesos
Si un proceso padre crea varios hijos, y esos hijos a su vez crean sus propios hijos, y así sucesivamente, se forma una estructura de árbol. Puedes visualizar esta estructura con el comando pstree en la terminal.

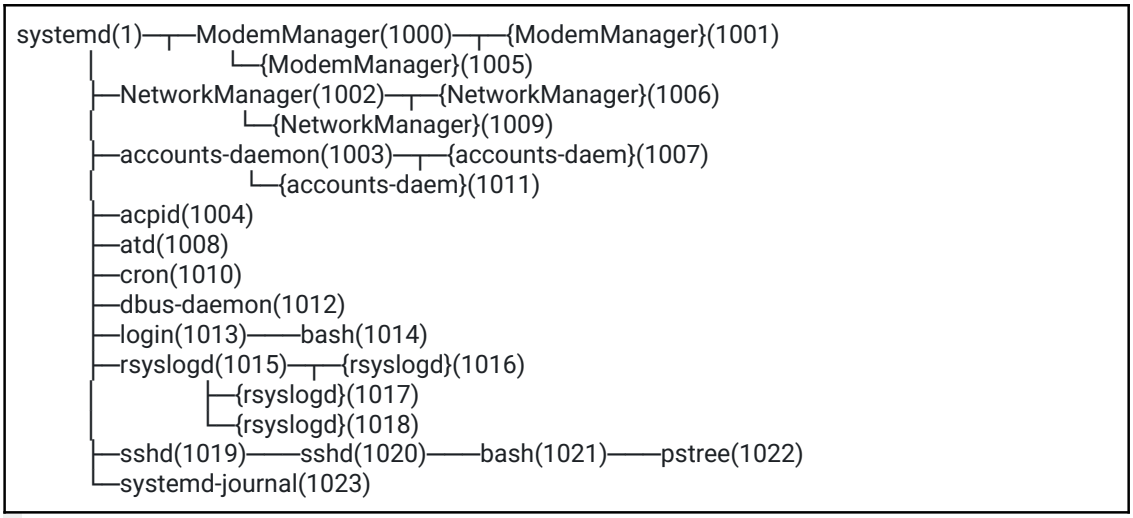
```
pstree -p
```

La opción -p hace que pstree muestre los PIDs (identificadores de proceso) de cada proceso. La salida de pstree será algo parecido a esto (los detalles exactos dependerán de tu sistema):

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu



En este ejemplo (simplificado):

- systemd(1) es el proceso raíz (PID 1).
- ModemManager, NetworkManager, accounts-daemon, etc., son procesos hijo de systemd.
- login es un proceso hijo de systemd, y bash (la terminal) es un proceso hijo de login.
- pstree (el comando que estamos ejecutando) es un proceso hijo de bash, que a su vez es hijo de sshd (el demonio de SSH), etc.

Cada proceso tiene un único padre (excepto systemd/init, que no tiene padre). Un proceso puede tener cero, uno o muchos hijos.

¿Qué Pasa Cuando un Proceso Padre Termina?

Si un proceso padre termina antes que sus hijos, los hijos se convierten en procesos "huérfanos". En ese caso, el sistema operativo hace que systemd (o init) "adopte" a esos hijos huérfanos. Es decir, systemd se convierte en el nuevo padre de esos procesos. Esto asegura que todos los procesos tengan un padre (excepto systemd).

Sesiones y Grupos de Procesos

Además de la jerarquía padre-hijo, los procesos también se organizan en *sesiones* y *grupos de procesos*. Esto sirve para gestionar mejor los recursos y para controlar cómo se envían las señales a los procesos

- **Sesión**: Una sesión es un conjunto de uno o más grupos de procesos. Normalmente, una sesión corresponde a una terminal. Cuando abres una terminal, se crea una nueva sesión. El proceso que crea la sesión se convierte en el líder de la sesión. Todos los procesos que crees en esa terminal (ejecutando comandos) pertenecerán a esa sesión.
- **Grupo de Procesos**: Dentro de una sesión, los procesos se pueden organizar en grupos. Cada grupo de procesos tiene un líder de grupo. Por defecto, cuando ejecutas un comando en la terminal, se crea un nuevo grupo de procesos para ese comando, y el proceso del comando se convierte en el líder del grupo.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.



¿Para qué sirven las sesiones y los grupos de procesos?

1. Control de Trabajos (Job Control): La shell (el programa que interpreta los comandos se escriben en la terminal) usa los grupos de procesos para gestionar los "trabajos" (jobs). Por ejemplo, puedes poner un trabajo en segundo plano (con `&`), traerlo a primer plano (con `fg`), suspenderlo (con Ctrl+Z), etc. Estas operaciones se aplican a todos los procesos del grupo.
2. Envío de Señales: Cuando pulsas Ctrl+C en la terminal, la señal de interrupción (SIGINT) se envía a todos los procesos del grupo de procesos que está en primer plano. Esto permite, por ejemplo, cancelar un comando que se está ejecutando. Si no existieran los grupos de procesos, la señal se enviaría a todos los procesos de la sesión, lo cual sería un poco patata.

3.1.3 Estados de un Proceso

¿Qué son los Estados de un Proceso?

Un proceso, a lo largo de su vida, no siempre está ejecutándose. Puede estar en diferentes *estados*, que reflejan lo que está haciendo o lo que está esperando en un momento dado. Imagina un trabajador en una fábrica: a veces está trabajando activamente, a veces está esperando materiales, y a veces ha terminado su trabajo. Los estados de un proceso son similares.

Los **estados básicos** de un proceso son:

1) Nuevo (New): El proceso acaba de ser creado. El sistema operativo está preparando las estructuras de datos necesarias para gestionarlo, pero todavía no está listo para ejecutarse.

2) Listo (Ready): El proceso está listo para ejecutarse. Tiene todo lo que necesita (memoria, recursos, etc.), pero está esperando su turno para usar la CPU. Hay muchos procesos listos, y el sistema operativo tiene que decidir cuál se ejecuta a continuación (esto se llama *planificación*).

3) Ejecución (Running): El proceso está usando la CPU en este momento. Sus instrucciones se están ejecutando.

4) Bloqueado (Blocked/Waiting): El proceso está esperando que ocurra algo antes de poder continuar. Por ejemplo:

- Está esperando a que se complete una operación de entrada/salida
- Está esperando a que otro proceso le envíe una señal.
- Está esperando a que un recurso esté disponible (p.ej: un semáforo).

Mientras está bloqueado, el proceso no puede usar la CPU, aunque quiera.

5) Terminado (Terminated): El proceso ha terminado de ejecutarse. Aunque ya no se ejecuta, el sistema operativo todavía guarda alguna información sobre él hasta que el proceso padre la recoja.

Transiciones entre Estados

Un proceso no se queda siempre en el mismo estado. Va cambiando de estado según lo que esté haciendo y lo que ocurra en el sistema. Estas transiciones son controladas por el *sistema operativo*.



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

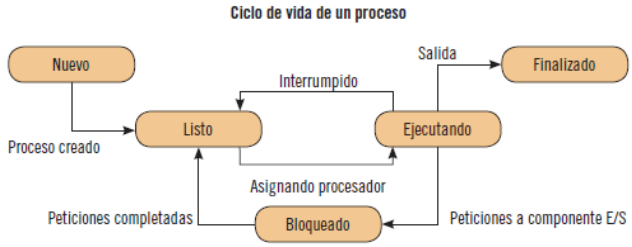


Fig. 1.1: Diagrama de estados de un proceso

Descripción de las transiciones:
Nuevo → Listo: Cuando el proceso se crea y se le asignan los recursos necesarios (excepto la CPU), pasa a estar listo para ejecutarse.

Listo → Ejecución: El *planificador* (scheduler) del sistema operativo decide qué proceso, de entre todos los que están listos, se ejecuta a continuación. El proceso elegido pasa al estado de ejecución.

Ejecución → Listo: El proceso puede perder la CPU por varias razones:

- Time Slice: El sistema operativo le da a cada proceso un tiempo limitado para usar la CPU. Cuando se acaba el tiempo, el proceso vuelve al estado listo, y otro proceso puede ejecutarse. Esto es muy importante para que todos los procesos tengan la oportunidad de ejecutarse y el sistema no se quede "colgado" con un solo proceso.
- Prioridad: Un proceso de mayor prioridad puede "desalojar" a un proceso de menor prioridad que se esté ejecutando.

Ejecución → Bloqueado: El proceso necesita esperar a que ocurra algo (una operación de entrada/salida, una señal, etc.). No puede continuar hasta que ese evento ocurra, así que pasa al estado bloqueado.

Bloqueado → Listo: Cuando ocurre el evento que el proceso estaba esperando, pasa del estado bloqueado al estado listo. No pasa directamente a ejecución; tiene que esperar su turno en la cola de procesos listos.

Ejecución → Terminado: El proceso ha terminado de ejecutarse (ha llegado al final de su código o ha sido terminado explícitamente).

CONCEPTO PELIGROSO (Interbloqueo)
Existe una situación peligrosa llamada interbloqueo (deadlock). Esto ocurre cuando dos o más procesos están bloqueados, esperando cada uno a que otro libere un recurso que necesita. Como todos están esperando, ninguno puede avanzar, y el sistema se queda "atascado". El sistema operativo tiene mecanismos para detectar y prevenir (o resolver) los interbloqueos, pero es un problema complejo.

Un proceso puede pasar por los estados de Listo, Ejecución y Bloqueado muchas veces durante su vida.

El sistema operativo se encarga de gestionar las transiciones entre estados y de asignar la CPU a los procesos listos mediante los planificadores

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

3.1.4 Estructuras de datos básicas para su gestión

Estructuras de Datos para la Gestión de Procesos

¿Por qué necesitamos estructuras de datos?

Las estructuras de datos permiten al sistema operativo:

- Llevar un registro de **todos** los procesos que se están ejecutando (o que están listos para ejecutarse, o bloqueados).
- Almacenar información importante sobre cada proceso (su estado, su PID, la memoria que está usando, etc.).
- Gestionar los recursos del sistema (CPU, memoria, archivos, etc.) eficientemente.
- Tomar decisiones sobre qué proceso debe ejecutarse después (planificación).

El Bloque de Control de Proceso (BCP)

La estructura de datos más importante para la gestión de procesos es el Bloque de Control de Proceso (BCP), también conocido como PCB (Process Control Block). Cada proceso tiene su propio BCP, y el sistema operativo usa el BCP para almacenar toda la información relevante sobre ese proceso.

¿Qué información contiene el BCP?

- **Identificador del Proceso (PID)**: Un número único que identifica al proceso.
- **Estado del Proceso**: Si el proceso está listo, en ejecución, bloqueado, etc.
- **Puntero al Proceso Padre (PPID)**: El PID del proceso que creó este proceso.
- **Prioridad del Proceso**: Un número que indica la importancia relativa del proceso
- **Contador de Programa (PC)**: La dirección de memoria de la siguiente instrucción que el proceso debe ejecutar. Esto es crucial para que el proceso pueda continuar ejecutándose correctamente después de interrumpirse.
- **Registros de la CPU**: Los valores de los registros de la CPU cuando el proceso no está en ejecución. Cuando el proceso vuelve a ejecutarse, el sistema operativo restaura los valores de los registros a partir del BCP.
- **Información de Memoria**: Información sobre la memoria que el proceso tiene asignada (direcciones de inicio y fin de las secciones de código, datos, pila, etc.).
- **Información de Entrada/Salida (E/S)**: Una lista de los archivos y dispositivos que el proceso tiene abiertos.
- **Información de Contabilidad**: Cuánto tiempo de CPU ha usado el proceso, cuánto tiempo lleva ejecutándose, etc.

El SO mantiene una tabla de procesos, que es una estructura de datos (normalmente un array o una lista enlazada) que contiene un BCP para cada proceso del sistema.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.


Máster en
Inteligencia Artificial y Ciencia de Datos
Impulsa tu perfil en IA y datos
Más info
• APRENDE A TRABAJAR CON DATOS REALES Y MODELOS PREDICTIVOS
• ENFOQUE PRÁCTICO CON CASOS DE NEGOCIO
• TOMA DECISIONES BASADAS EN ANALÍTICA AVANZADA
• SALIDAS EN DATA, IA Y TRANSFORMACIÓN DIGITAL

Ejemplo de BCP:

Campo	Descripción
PID	1234
Estado	Listo
PPID	1000
Prioridad	5
Contador de Programa	0x40001000
Registros de la CPU	(Valores de los registros)
Memoria Asignada	(Información sobre las regiones de memoria)
Archivos Abiertos	(Lista de descriptores de archivo)

Cada proceso en ejecución tendría su propia "ficha" como esta.

Gestión de Descriptores de Archivos (Repaso del Módulo 2)
Cuando un programa necesita interactuar con un archivo (leerlo, escribirlo, etc.), o con otros recursos como conexiones de red (sockets) o tuberías (pipes), utiliza descriptores de archivo. Un descriptor de archivo es simplemente un número entero (no negativo) que el sistema operativo usa para identificar un archivo abierto o un recurso de E/S (Módulo 2).

Tablas de Descriptores de Archivos
Para gestionar los descriptores de archivos, el sistema operativo utiliza dos tablas principales:
1) **Tabla de Archivos Abiertos del Sistema** (Global) (Módulo 2): Esta tabla es compartida por todos los procesos del sistema. Contiene una entrada por cada archivo que está abierto en todo el sistema.
2) **Tabla de Descriptores de Archivos por Proceso**: Cada proceso tiene su propia tabla de descriptores de archivos. Esta tabla es privada del proceso.

Otras Estructuras de Datos Importantes
Además del BCP y las tablas de descriptores de archivos, el sistema operativo utiliza otras estructuras de datos para gestionar los procesos y los recursos del sistema. Aquí veremos tres ejemplos importantes:

1. Tabla de Procesos
La tabla de procesos es una estructura de datos global (hay una sola tabla para todo el sistema) que contiene una entrada para cada proceso que existe en el sistema. Cada entrada en la tabla de procesos es, precisamente, el *Bloque de Control de Proceso (BCP)* de un proceso.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

¿Para qué sirve la tabla de procesos?

- Lista completa de procesos: Permite al sistema operativo tener una visión general de todos los procesos que existen, independientemente de su estado.
- Acceso rápido a la información del proceso*:* El sistema operativo puede acceder rápidamente al BCP de cualquier proceso usando su PID como índice en la tabla (o como clave de búsqueda).
- Gestión de recursos*:* La tabla de procesos, a través de los BCPs, permite al sistema operativo saber qué recursos (memoria, archivos, etc.) tiene asignado cada proceso.
- Planificación*:* El planificador del sistema operativo usa la información de la tabla de procesos (estado, prioridad, etc.) para decidir qué proceso debe ejecutarse a continuación.

Analogía sencilla: La tabla de procesos es como el registro de todos los empleados de una empresa. Cada empleado tiene una ficha (el BCP) con su información y la empresa usa ese registro para gestionar a todos sus empleados.

2. Tabla de Páginas

La tabla de páginas es una estructura que se usa para la gestión de la memoria virtual. (Módulo 5) La memoria virtual es una técnica que permite a los programas usar más memoria de la que físicamente existe en el ordenador. Esto se logra combinando la memoria RAM con el almacenamiento en disco.

¿Cómo funciona la memoria virtual? (Ayuda para el Módulo 5)

La memoria que "ve" un proceso (su *espacio de direcciones virtuales*) se divide en trozos de tamaño fijo llamados *páginas*.

La memoria RAM física también se divide en trozos del mismo tamaño, llamados *marcos de página* (o *frames*).

No todas las páginas de un proceso necesitan estar en la RAM al mismo tiempo. Algunas pueden estar en el disco (en un área especial llamada *espacio de intercambio* o *swap*).

La tabla de páginas es un "mapa" que indica, para cada página virtual de un proceso, dónde se encuentra:

- Si la página está en la RAM, la tabla de páginas indica en qué marco de página físico se encuentra.
- Si la página está en el disco, la tabla de páginas indica en qué lugar del disco está.
- Puede indicar que la página no está cargada ni en RAM ni en disco.

Cuando un proceso intenta acceder a una dirección de memoria, el SO consulta la tabla de páginas para traducir la *dirección virtual* a una *dirección física*

- Si la página está en la RAM, la traducción es directa.
- Si la página está en el disco, se produce un *fallo de página* (page fault). El SO suspende el proceso, carga la página del disco a la RAM actualiza la tabla de páginas, y luego reanuda el proceso.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu

3. Colas de Planificación

Las **colas de planificación** son estructuras de datos que el SO usa para organizar los procesos que están en estado *listo*. El *planificador* (scheduler) del sistema operativo es el componente responsable de decidir qué proceso, de entre todos los que están listos, se ejecuta a continuación.

3.2 Planificación

Planificación y Multitarea: El Arte de Hacer Varias Cosas a la Vez

¿Qué es la Multitarea?

La multitarea es la capacidad de un sistema operativo para ejecutar varios programas (o mejor dicho, varios *procesos*) aparentemente al mismo tiempo. (Un procesador solo puede ejecutar una instrucción a la vez). La multitarea se consigue alternando rápidamente entre los diferentes procesos, dando la ilusión de que todos se ejecutan simultáneamente.

¿Cómo funciona la Multitarea?

La clave de la multitarea está en el *planificador* (scheduler) del sistema operativo. El planificador es un componente del sistema operativo que decide qué proceso debe ejecutarse en cada momento. Para ello, utiliza varios algoritmos y criterios, como:

- *Prioridad de los procesos:* Algunos procesos son más importantes que otros. Por ejemplo, un proceso que controla el sistema de frenos de un coche es mucho más importante que un proceso que reproduce música.
- *Tiempo de espera:* Si un proceso lleva mucho tiempo esperando para usar la CPU, el planificador puede darle prioridad
- *Tipo de proceso:* Los procesos interactivos (con los que interactúa directamente el usuario) suelen tener prioridad sobre los procesos en segundo plano.
- *Tiempo de CPU ya consumido*: Para evitar que un proceso monopolice la CPU.

El planificador divide el tiempo de la CPU en pequeñas porciones llamadas *rodajas de tiempo* (time slices) o *cuantos* (quantum). A cada proceso se le asigna una rodaja, y cuando se agota el tiempo, o el proceso se bloquea porque necesita esperar a un evento externo (como leer algo de un archivo), se cambia el contexto a otro proceso.

Cambio de Contexto: Guardar el estado del proceso actual y cargar el estado del siguiente proceso. Este cambio es rápido, pero no instantáneo. Tiene un coste.

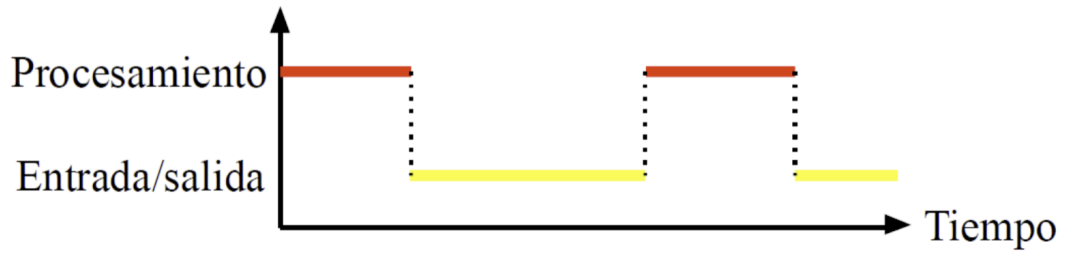


Fig 1.1: Diagrama de multitarea

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu


Máster en
Inteligencia Artificial
& Data Management
Gestiona datos como activo estratégico
Más info
• APRENDE CON PROFESIONALES EN ACTIVO
• METODOLOGÍAS COMO DESIGN THINKING Y CASOS REALES
• PERFIL CLAVE EN ORGANIZACIONES DATA-DRIVEN

El Proceso Nulo
¿Qué pasa cuando no hay ningún proceso que necesite usar la CPU? ¿Se apaga el procesador? No. En lugar de eso, el SO ejecuta el *proceso nulo* (idle process).

El proceso nulo es un proceso muy simple. Su única función es mantener la CPU ocupada cuando no hay ningún otro proceso que necesite ejecutarse.

¿Por qué es necesario el proceso nulo?

- **Mantiene el sistema estable**: Algunos procesadores pueden tener problemas si se quedan inactivos durante mucho tiempo. El proceso nulo evita esto.
- **Simplifica la planificación**: El planificador del sistema operativo siempre tiene al menos un proceso para ejecutar (el proceso nulo), lo que simplifica su lógica.
- **Medición del tiempo de inactividad**: El tiempo que la CPU pasa ejecutando el proceso nulo es una medida del tiempo de inactividad del sistema.

En Linux, el proceso nulo suele tener el PID 0. Se crea automáticamente.

Ventajas de la Multitarea

- **Mejor aprovechamiento de la CPU***:* Mientras un proceso espera (por ejemplo, a que se complete una operación de entrada/salida), otro proceso puede usar la CPU. Esto reduce el tiempo de inactividad de la CPU y aumenta la eficiencia del sistema.
- **Mejor experiencia de usuario***:* La multitarea permite a los usuarios ejecutar varios programas al mismo tiempo y cambiar entre ellos rápidamente. Esto hace que el ordenador sea mucho más útil y versátil.
- **Modularidad:** La multitarea facilita la creación de programas modulares, dividiendo tareas complejas en procesos más pequeños e independientes.
- **Facilita la programación concurrente**: Permite a los desarrolladores crear aplicaciones que realicen varias tareas al mismo tiempo.

Grado de Multiprogramación
El **grado de multiprogramación** (o *grado de multitarea*) se refiere al número de procesos que están activos en el sistema en un momento dado (están en memoria).

Un mayor grado de multiprogramación puede mejorar el aprovechamiento de la CPU, pero también aumenta la sobrecarga del sistema operativo. Hay un límite a la cantidad de procesos que un sistema puede manejar eficientemente. Si hay demasiados procesos, el sistema puede volverse lento o inestable (esto se conoce como "thrashing").

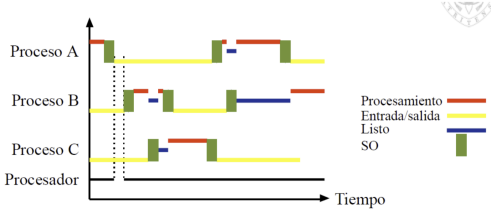


Fig 1.2: Grado de multiprogramación

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3.2.1 Concepto de planificador

¿Qué es el Planificador?
El **planificador** (*schedule*r) es un componente del SO. Es el responsable de decidir *qué proceso* se ejecuta en la CPU en cada momento.

Objetivos del planificador (Importante):

- **Maximizar el uso de la CPU***:* Que la CPU esté ocupada todo lo posible.
- **Minimizar el tiempo de espera**
- **Maximizar el rendimiento***:* Que se complete la mayor cantidad de trabajo posible en un tiempo determinado.
- **Ser justo***:* Que todos los procesos tengan una oportunidad de ejecutarse.
- **Responder rápidamente***:* Que los procesos interactivos respondan rápidamente a las acciones del usuario.

Niveles de Planificación
La planificación no se realiza de una sola vez, existen diferentes niveles:

- **Planificación a largo plazo**: Decide qué procesos se *admiten* en el sistema. Controla el grado de multiprogramación. No todos los SOs tienen.
- **Planificación a medio plazo**: Se encarga de *suspender* y *reanudar* procesos. Esto se hace para controlar el grado de multiprogramación y liberar memoria si es necesario. Es como un intercambio (swapping) de procesos entre la memoria principal y el disco.
- **Planificación a corto plazo (o de CPU)**: Es el planificador más importante, y el que normalmente llamamos simplemente "planificador". Es el que decide qué proceso, de entre todos los que están *listos*, se ejecuta a continuación en la CPU.

Interrupciones
El planificador no siempre decide cuándo cambiar de proceso. A veces, el proceso actual es *interrumpido*.
Una interrupción es una señal que indica que ha ocurrido un evento que requiere atención inmediata. Las interrupciones pueden ser causadas por:

Hardware:
Un dispositivo de E/S (por ejemplo, el teclado, el ratón, el disco duro) ha terminado una operación o necesita atención.
El temporizador (timer) del sistema ha expirado

Software:

- Un proceso ha realizado una llamada al sistema (system call).
- Un proceso ha cometido un error (por ejemplo, una división por cero o un acceso a memoria no válido).

Cuando ocurre una interrupción, el sistema operativo:

1. *Detiene* el proceso que se estaba ejecutando.
2. *Guarda* el estado del proceso actual (en su BCP).
3. *Cambia* al modo supervisor (o modo kernel).

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

4. *Ejecuta* una rutina especial, llamada *manejador de interrupciones* (interrupt handler) o *rutina de servicio de interrupción* (interrupt service routine, ISR), para atender la interrupción.
5. Una vez que la interrupción ha sido atendida, el sistema operativo puede decidir:
    - *Reanudar* el proceso que fue interrumpido.
    - *Ejecutar* otro proceso (si el planificador así lo decide).

RTI (Return from Interrupt): Después de que el manejador de interrupciones termina, se ejecuta una instrucción especial llamada *RTI* (Return from Interrupt). Esta instrucción restaura el estado del proceso que fue interrumpido (a partir de la información guardada en el BCP) y devuelve el control a ese proceso.

Activación del Planificador

El planificador se activa en las siguientes situaciones:

- Cuando un proceso se crea o termina.
- Cuando un proceso se bloquea (esperando una operación de E/S, por ejemplo).
- Cuando un proceso pasa de bloqueado a listo (por ejemplo, cuando la operación de E/S que estaba esperando termina).
- Cuando ocurre una interrupción del temporizador (en sistemas con planificación por tiempo compartido, como Round Robin).
- Cuando ocurre una interrupción de hardware (teclado, ratón, red...).
- Cuando un proceso realiza una llamada al sistema.

Cambio de Contexto (Context Switch)

Cuando el planificador decide cambiar de un proceso a otro, se produce un *cambio de contexto* (context switch). El cambio de contexto es el proceso de guardar el estado del proceso que se estaba ejecutando y cargar el estado del proceso que se va a ejecutar.

Pasos de un Cambio de Contexto:

1. Guardar el contexto del proceso actual: El sistema operativo guarda el estado actual del proceso que se estaba ejecutando en su BCP. Esto incluye:
    - El valor del contador de programa (PC).
    - Los valores de los registros de la CPU.
    - El estado de la pila.
    - Otra información relevante (estado de los archivos abiertos, etc.).
2. Seleccionar el siguiente proceso: El planificador elige el siguiente proceso según su algoritmo de planificación.
3. Cargar el contexto del nuevo proceso: El sistema operativo carga el estado del proceso seleccionado desde su BCP. Esto incluye:
    - Restaurar el valor del contador de programa.
    - Restaurar los valores de los registros de la CPU.
    - Ajustar las estructuras de datos de gestión de memoria
4. Transferir el control a la CPU: Se reanuda la ejecución en la instrucción indicada por el PC del nuevo proceso.

El cambio de contexto es una operación *costosa* en términos de tiempo. Mientras se realiza el cambio de contexto, la CPU no está ejecutando código útil de ningún proceso. Por lo tanto, los sistemas operativos intentan minimizar el número de cambios de contexto.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu



3.2.2 Estrategias de Planificación

¿Qué es la Planificación de Procesos?
La *planificación de procesos* es la actividad del sistema operativo que consiste en decidir qué proceso, de entre todos los que están listos para ejecutarse, va a utilizar la CPU a continuación. El *planificador* (o *scheduler*) realiza esta tarea.

Concepto:
Expropiativo vs. No Expropiativo:

- *No expropiativo:* Una vez que un proceso empieza a usar la CPU, sigue usándola hasta que termina o hasta que se bloquea voluntariamente
- *Expropiativo:* El sistema operativo puede quitarle la CPU a un proceso que se está ejecutando y dársela a otro proceso

Algoritmos de Planificación
1. FCFS (First-Come, First-Served - Primero en Llegar, Primero en Ser Servido)
Los procesos se ejecutan en el orden en que llegan a la cola de listos.
Características

- *No expropiativo.*
- *Simple* de implementar.
- *Justo* en el sentido de que respeta el orden de llegada.

Ventaja: Simple.
Desventaja: Puede provocar tiempos de espera muy largos para los procesos cortos si llegan después de procesos largos. No es muy eficiente.

2. SJF (Shortest Job First - El Trabajo Más Corto Primero)
Se ejecuta primero el proceso que tiene el *tiempo de CPU estimado más corto.*
Características

- Puede ser *expropiativo* o *no expropiativo*.
- *Minimiza* el tiempo de espera promedio.

Ventaja: Minimiza el tiempo de espera promedio.
Desventajas:

- Es *difícil* saber de antemano cuánto tiempo de CPU necesitará un proceso.
- Puede provocar *inanición* (starvation) de los procesos largos (si siempre llegan procesos más cortos, los largos nunca se ejecutan).

3. Planificación por Prioridades (No Expropiativo)
A cada proceso se le asigna una *prioridad*. El planificador siempre elige el proceso de *mayor prioridad* que esté listo para ejecutarse.
Características

- Puede ser *expropiativo* o *no expropiativo* (en este caso, no expropiativo).

Ventajas:

- Permite dar preferencia a los procesos más importantes.

Desventajas:

- Puede provocar *inanición* de los procesos de baja prioridad.
- Es necesario definir un criterio para asignar las prioridades.



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.


ROTOTOM
SUNSPLASH
Lia Kali, Protoje,
Kybba, MAJOR LAZER,
QUEEN OMEGA,
SHENSEEA...
Y AÚN HAY MÁS
El festival de reggae más grande
de Europa vuelve a Benicàssim.
¿Ya tienes tu entrada?
Pillar mi entrada
ROTOTOM.COM
16-22/08 2026 | benicàssim spain
CASTELLÓN

4. Round Robin (RR)
Es un algoritmo *expropiativo*. A cada proceso se le asigna un *cuanto* (quantum) de tiempo de CPU. Si el proceso no termina en ese tiempo, se le quita la CPU y se pone al final de la cola de listos. El planificador elige el siguiente proceso de la cola.
Características:

- *Expropiativo.*
- ***Justo*, todos los procesos reciben tiempo de CPU.**
- ***Fácil* de implementar.**

Ventajas:

- *Justo*: Todos los procesos reciben tiempo de CPU.
- *Bueno para procesos interactivos*: Responde rápidamente a las acciones del usuario.

Desventajas:

- Si el cuanto es muy pequeño, hay *demasiados cambios de contexto*, lo que reduce la eficiencia.
- Si el cuanto es muy grande, se parece a FCFS.

5. SRTF (Shortest Remaining Time First - Tiempo Restante Más Corto Primero)
Es la versión *expropiativa* de SJF. El planificador elige el proceso que tiene el *tiempo restante de CPU más corto*. Si llega un nuevo proceso con un tiempo restante menor que el proceso que se está ejecutando actualmente, el proceso actual se expropia y se ejecuta el nuevo proceso.

Características

- *Expropiativo*
- *Minimiza* el tiempo de espera promedio.

Ventajas:

- *Minimiza* el tiempo de espera promedio.

Desventajas:

- Es difícil saber de antemano el tiempo restante de CPU.
- Puede provocar *inanición*.

6. Planificación Expropiativa con Prioridades
Similar a la planificación por prioridades no expropiativa, pero en este caso, si llega un proceso con mayor prioridad que el proceso que se está ejecutando actualmente, el proceso actual se expropia y se ejecuta el nuevo proceso.

Características: *Expropiativo.*
Mismas ventajas y desventajas que la planificación por prioridades no expropiativa, pero con una respuesta más rápida a los procesos de alta prioridad.

7. Colas Multinivel con Realimentación
Este es un algoritmo más complejo, que combina ideas de varios de los algoritmos anteriores. Se utilizan varias colas de listos, cada una con una prioridad diferente.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu

Funcionamiento (de ejemplo, el SO puede tener sus propias reglas):

- Los procesos nuevos entran en la cola de más alta prioridad.
- Si un proceso agota su cuanto de tiempo (como en Round Robin) sin bloquearse, se le baja a una cola de menor prioridad.
- Si un proceso se bloquea (por ejemplo, por una operación de E/S), cuando vuelve a estar listo, puede subir a una cola de mayor prioridad (o quedarse en la misma, dependiendo de la implementación específica). Esto se llama "realimentación" (feedback).
- Dentro de cada cola, se puede usar un algoritmo diferente (por ejemplo, Round Robin en las colas de alta prioridad, FCFS en las de baja prioridad).

Objetivo:

- Favorecer a los procesos cortos e interactivos (que suelen bloquearse con frecuencia). Estos procesos se quedarán en las colas de alta prioridad.
- Evitar la inanición de los procesos largos (que eventualmente llegarán a la cola de más baja prioridad, donde se ejecutarán hasta el final).

Reglas: Las reglas de este algoritmo son totalmente dependientes de las necesidades, para nuestra asignatura, el enunciado debe dar las reglas necesarias para poder resolver el ejercicio, es decir:

- Número de colas.
- Algoritmo de planificación de cada cola.
- Cómo se decide a qué cola entra un proceso nuevo.
- Cómo se decide cuándo un proceso sube o baja de cola (la "realimentación").

Ventajas:

- *Flexible*: Se puede adaptar a diferentes tipos de procesos.
- *Buen tiempo de respuesta* para procesos interactivos.
- *Evita la inanición*.

Desventajas:

- *Complejo* de implementar y ajustar.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

SHÔKO MADRID
EL EXAMEN SE APRUEBA ESTUDIANDO,
LA CARRERA SE VIVE SALIENDO.
Dale el último empujón a los apuntes y vente.
Pilla tu entrada
Nosotros ponemos la música y las copas para cuando termines el tema. Remata la faena y asegúrate la entrada ya, que vuelan.
Domingo
Sábado
Viernes
Jueves
Lunes

3.3 Threads

Hilos (Threads) vs. Procesos

¿Qué son los Hilos?

Hasta ahora, hemos hablado de procesos como programas en ejecución. Pero dentro de un proceso, puede haber varias *hebras de ejecución*, o *hilos* (threads, en inglés). Un hilo es una secuencia de instrucciones que se puede ejecutar concurrentemente con otros hilos dentro del mismo proceso.

Características de los Hilos:

- *Comparten el mismo espacio de direcciones:* Esto significa que todos los hilos de un proceso pueden acceder a las mismas variables globales, al mismo montículo (heap), a los mismos archivos abiertos, etc.
- *Son más "ligeros" que los procesos:* Crear un hilo nuevo es más rápido y consume menos recursos que crear un proceso nuevo (porque no hay que duplicar todo el espacio de direcciones).
- *Comunicación más fácil:* Como los hilos comparten memoria, pueden comunicarse entre sí de forma más sencilla y rápida que los procesos (que necesitan mecanismos especiales de comunicación entre procesos, como tuberías o memoria compartida).
- *Mayor riesgo de errores:* Como los hilos comparten memoria, si no se tiene cuidado, pueden "pisarse" los datos unos a otros, provocando errores difíciles de depurar (esto se llama *condición de carrera* o *race condition*).

Diferencias Clave: Hilos vs. Procesos

Característica	Hilos (Threads)	Procesos
Espacio de direcciones	Compartido (dentro del mismo proceso)	Independiente
Recursos (memoria, archivos, etc.)	Compartidos	Propios
Creación	Más rápida y ligera	Más lenta y pesada
Comunicación	Más fácil (memoria compartida)	Más compleja (mecanismos especiales)
Seguridad	Menos seguros (riesgo de condiciones de carrera)	Más seguros (aislamiento)
PID	Mismo PID que el proceso padre	PID diferente del padre

Sección Extra de Ayuda 1: ¿Es lo mismo un Hilo que un fork()?

No. fork() crea un *proceso* nuevo, mientras que los hilos son unidades de ejecución dentro de un proceso. Son conceptos relacionados, pero diferentes.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- fork():
    - Crea un *proceso hijo*, que es una copia (casi) idéntica del proceso padre.
    - El padre y el hijo tienen espacios de direcciones independientes. Si el hijo modifica una variable, el padre no ve el cambio, y viceversa.
    - La comunicación entre padre e hijo requiere mecanismos especiales (tuberías, memoria compartida, señales, etc.).
    - El hijo tiene su propio PID.
- Hilos:
    - Se crean dentro*de un proceso existente.
    - Todos los hilos de un proceso comparten el mismo espacio de direcciones. Si un hilo modifica una variable global, todos los demás hilos ven el cambio.
    - La comunicación entre hilos es más sencilla (pueden usar variables globales), pero requiere sincronización para evitar condiciones de carrera.
    - Los hilos comparten el mismo PID del proceso al que pertenecen.

¿Cómo detectar si es un Hilo o un fork()?
La principal diferencia que puedes observar desde dentro del programa es el espacio de direcciones:

- Hilos: Si modificas una variable global en un hilo, todos los demás hilos del mismo proceso verán el cambio.
- fork(): Si modificas una variable global en el proceso hijo, el proceso padre no verá el cambio (y viceversa).

Otra diferencia clave es el PID:

- Hilos: Todos los hilos de un proceso tienen el *mismo* PID (el PID del proceso).
- fork(): El proceso hijo tiene un PID *diferente* del PID del padre.

Sección Extra de Ayuda 2: Errores Comunes en el Uso de Hilos y fork()

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

Errores a Evitar

Condiciones de carrera (Race Conditions): Este es el error más común cuando se usan hilos. Ocurre cuando dos o más hilos acceden a la misma variable compartida al mismo tiempo, y al menos uno de ellos la modifica. El resultado final depende del orden en que se ejecuten los hilos, y puede ser impredecible. La solución es usar mecanismos de *sincronización*, como mutexes (mutual exclusion), semáforos, o variables condicionales.

No comprobar el valor de retorno de fork(): fork() puede fallar. Siempre debes comprobar si el valor de retorno es -1, y si lo es, manejar el error (por ejemplo, imprimiendo un mensaje de error y saliendo del programa).

Olvidar el NULL al final de la lista/vector de argumentos en exec: Si usas execl, execlp, execle, el último argumento debe ser `(char *)NULL`. Si usas execv, execvp, execvpe, el último elemento del array de argumentos debe ser `NULL`. Si te olvidas de esto, el programa puede comportarse de forma extraña (o incluso fallar).

No cerrar descriptores de archivo en el hijo después de fork(): Si el padre tiene archivos abiertos, el hijo hereda copias de los descriptores de archivo. Si el hijo no va a usar esos archivos, debe cerrarlos. De lo contrario, podrías tener fugas de descriptores de archivo, o comportamientos inesperados (por ejemplo, si el padre y el hijo intentan escribir en el mismo archivo al mismo tiempo).

No esperar a los hijos (procesos zombie): Si el padre no llama a wait() (o waitpid()) para esperar a que sus hijos terminen, los hijos se convertirán en procesos zombie.

Interbloqueos (Deadlocks): Esto puede ocurrir tanto con hilos como con procesos. Un interbloqueo ocurre cuando dos o más procesos/hilos están esperando a que otro libere un recurso que necesitan, y ninguno puede avanzar. Los interbloqueos son muy difíciles de depurar

3.3.1 Concepto de Thread

Ya hemos introducido la idea de los hilos (threads) al compararlos con los procesos. Ahora vamos a profundizar un poco más en qué son exactamente los hilos y cómo funcionan.

Definición: Un *hilo* (thread) es la unidad básica de ejecución dentro de un proceso. Un hilo es, por lo tanto, una secuencia de instrucciones que se ejecuta de forma independiente, pero dentro del contexto de un proceso.

Diferencia con un programa:
Un programa es un ente estático, son las instrucciones para que la máquina haga algo, pero no está "vivo". Un hilo es la ejecución de parte de un programa, por lo que sí que está "vivo", es dinámico.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

knoweats!
Estudiar, dormir y comer bien
Elige las tres. Con Knoweats lo tienes fácil: comida casera lista para que te centres en lo demás.
Comer bien



20€ DE DESCUENTO
con el código 20ESTUDIO

Características Clave de los Hilos (Teoría):

- *Comparten el espacio de direcciones:* Esto es lo más importante. Todos los hilos dentro de un mismo proceso comparten:
  - La memoria (variables globales, montículo).
  - Los archivos abiertos.
  - El código del programa.
  - Otras variables de entorno.
- *Tienen su propio contador de programa (PC):* Cada hilo tiene su propio PC, que indica la siguiente instrucción a ejecutar. Esto permite que cada hilo ejecute una parte diferente del código del programa de forma independiente.
- *Tienen su propia pila:* Cada hilo necesita su propia pila para almacenar las variables locales de las funciones que llama, las direcciones de retorno, etc.
- *Tienen su propio conjunto de registros*: Cada hilo tiene una copia de los registros de la CPU, para no interferir con los demas hilos.
- *Son más "ligeros" que los procesos:* Crear un hilo es más rápido y consume menos recursos que crear un proceso, porque no hay que duplicar todo el espacio de direcciones.
- *Comparten el PID del proceso padre*: A diferencia de los procesos creados con fork, que tienen cada uno un PID diferente, los hilos de un mismo proceso tienen todos el mismo PID

Sección Extra de Ayuda 1: ¿Por Qué Usar Hilos?

Paralelismo: En sistemas con múltiples procesadores (o núcleos), los hilos pueden ejecutarse realmente en paralelo, lo que puede acelerar mucho la ejecución de un programa.

Concurrencia: Incluso en sistemas con un solo procesador, los hilos permiten que un programa realice varias tareas "al mismo tiempo" (alternando rápidamente entre ellas). Esto es especialmente útil para programas que realizan operaciones de entrada/salida, que pueden tardar mucho tiempo.

Compartición de recursos: Como los hilos comparten memoria, es muy fácil y eficiente que se comuniquen y compartan datos entre ellos.

Responsividad: En aplicaciones interactivas (como un editor de texto o un navegador web), los hilos permiten que la aplicación siga respondiendo a las acciones del usuario mientras realiza otras tareas en segundo plano (como guardar el documento o descargar una página web)

Condiciones de Carrera

Una condición de carrera (race condition) ocurre cuando dos o más hilos acceden a la misma variable compartida al mismo tiempo, y al menos uno de ellos la modifica. El resultado final depende del orden en que se ejecuten los hilos, y puede ser impredecible.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3.3.2 Estructura de una Aplicación Multithread

Paralelización con Hilos

Una de las principales ventajas de usar hilos es la capacidad de *paralelizar* tareas. Esto significa que puedes dividir un trabajo grande en varias partes más pequeñas y hacer que cada hilo se encargue de una parte. Si tienes varios núcleos de CPU, los hilos pueden ejecutarse realmente al mismo tiempo, lo que acelera el proceso.

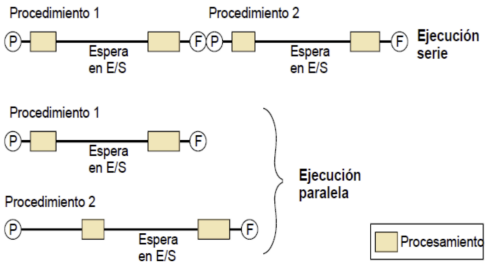


Fig. 1.1: Paralelización usando hilos

Alternativas al Diseño Multihilo (Teoría)

Realmente los hilos no siempre son la mejor opción. Hay situaciones en las que otras alternativas pueden ser más adecuadas.

1. Proceso de un Solo Hilo

Un proceso de un solo hilo es el modelo de programación "tradicional". El programa tiene un único hilo de ejecución, y todas las tareas se realizan secuencialmente, una después de otra.

Ventajas:

- *Simplicidad:* Es el modelo más sencillo de entender y de programar. No hay que preocuparse por la sincronización entre hilos ni por las condiciones de carrera.

Desventajas

- *Bajo rendimiento*: En tareas que podrían paralelizarse, un solo hilo no aprovecha los recursos de los procesadores multinúcleo.
- *Poca capacidad de respuesta*: Si el hilo realiza una operación bloqueante (como leer de un archivo), todo el programa se detiene hasta que la operación se completa.

¿Cuándo usarlo?:

- Para programas muy sencillos que no requieren paralelismo ni concurrencia.
- **Cuando la simplicidad es más importante que el rendimiento.**
- **En sistemas que no soportan hilos.**

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

2. Múltiples Procesos (sin hilos)
En lugar de usar hilos dentro de un mismo proceso, se pueden usar múltiples procesos independientes para realizar tareas en paralelo. Cada proceso tiene su propio espacio de direcciones, por lo que no hay riesgo de condiciones de carrera.

Ventajas:
- *Aislamiento:* Si un proceso falla, no afecta a los demás procesos.
- *Seguridad:* Los procesos no pueden acceder directamente a la memoria de otros procesos (a menos que se usen mecanismos especiales de comunicación entre procesos).

Desventajas:
- *Mayor sobrecarga:* Crear y gestionar procesos es más costoso (en términos de tiempo y recursos) que crear y gestionar hilos.
- *Comunicación más compleja:* Como los procesos no comparten memoria, la comunicación entre ellos es más complicada. Hay que usar mecanismos especiales, como tuberías (pipes), colas de mensajes, memoria compartida o sockets.

¿Cuándo usarlo?
- Cuando el aislamiento entre las tareas es importante (por seguridad o estabilidad).
- Cuando las tareas no necesitan compartir mucha información entre ellas.
- **En sistemas distribuidos (varios ordenadores trabajando juntos).**

3. Llamadas al Sistema Bloqueantes vs. No Bloqueantes
Cuando un programa necesita realizar una operación de entrada/salida (E/S), como leer de un archivo, escribir en un archivo, enviar o recibir datos por la red, etc., tiene dos opciones:
- Llamadas bloqueantes (síncronas): El proceso se *bloquea* (se detiene) hasta que la operación de E/S se completa. Durante ese tiempo, el proceso no puede hacer nada más.
- Llamadas no bloqueantes (asíncronas): El proceso inicia la operación de E/S, pero *no espera* a que se complete. El proceso puede seguir haciendo otras cosas. El sistema operativo avisará al proceso cuando la operación de E/S haya terminado (mediante una señal, un callback, etc.).

Ventajas de las llamadas no bloqueantes:
- *Mayor eficiencia:* El proceso no pierde tiempo esperando a que se completen las operaciones de E/S. Puede seguir haciendo otras cosas.
- *Mayor capacidad de respuesta:* En aplicaciones interactivas, esto permite que la aplicación siga respondiendo a las acciones del usuario mientras se realizan operaciones de E/S en segundo plano.

Desventajas de las llamadas no bloqueantes:
- *Mayor complejidad:* La programación con llamadas no bloqueantes suele ser más compleja que con llamadas bloqueantes, ya que hay que gestionar los eventos de E/S de forma asíncrona.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

CARPE DIEM
Organiza tu futuro:
estudia hoy para destacar mañana.
En Carpe Diem te esperan cursos adaptados a ti.
Una forma fácil y real de avanzar profesionalmente.
¡Quiero formarme!
Acreditados
Económicos
Desde casa
Elige tu curso



3.3.3 Implementación de los Threads. Biblioteca POSIX Threads

La Biblioteca Pthreads

Para trabajar con hilos en C en sistemas tipo Unix (Linux, macOS, etc.), se utiliza la biblioteca *Pthreads* (POSIX Threads). POSIX es un estándar que define una interfaz común para sistemas operativos, y Pthreads es la parte de ese estándar que se refiere a los hilos. Pthreads proporciona un conjunto de funciones para crear, gestionar y sincronizar hilos.
Importante: Para usar pthreads, necesitas:

- *Incluir el archivo de cabecera pthread.h*: Este archivo contiene las declaraciones de las funciones y tipos de datos de Pthreads.
- *Compilar con la opción -pthread*: Esto le indica al compilador que enlace tu programa con la biblioteca Pthreads.

Funciones Clave de Pthreads

Vamos a ver las funciones más importantes de Pthreads para crear y gestionar hilos:

1. pthread_create

Esta función se usa para crear un nuevo hilo.
Sintaxis:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);
```

Argumentos:

- pthread_t *thread: Un puntero a una variable de tipo pthread_t. pthread_t es un tipo de dato especial que se usa para identificar a un hilo. pthread_create escribirá el identificador del nuevo hilo en esta variable.
- const pthread_attr_t *attr: Un puntero a una estructura de atributos del hilo. Esta estructura permite especificar características especiales del hilo, como su tamaño de pila o su política de planificación. Si pasas NULL, se usan los atributos por defecto (que es lo más común para empezar).
- void *(*start_routine) (void *): Esta es la parte más importante. Es un puntero a la *función que el nuevo hilo ejecutará*. Es decir, aquí le dices a pthread_create "quiero que el nuevo hilo empiece a ejecutar esta función".
    - La función debe tener esta forma: void *(*start_routine) (void *).
        - Debe devolver un void * (un puntero genérico).
        - Debe recibir un void * como argumento.
        - El nombre start_routine es solo un nombre genérico; puedes poner el nombre que quieras a tu función.
- void *arg: Un puntero al *argumento* que se le pasará a la función start_routine. Puede ser un puntero a cualquier tipo de dato, o incluso NULL si no necesitas pasar ningún argumento.

Valor de retorno:

- pthread_create devuelve 0 si tiene éxito, o un código de error distinto de cero si falla. Es *muy importante* comprobar el valor de retorno para asegurarte de que el hilo se creó correctamente.

WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Compilación:

```
gcc -pthread -o ejemplo_hilo ejemplo_hilo.c
```

2. pthread_join

La función pthread_join se usa para *esperar* a que un hilo termine. Es como un "punto de encuentro" entre el hilo que llama a pthread_join (normalmente el hilo principal) y el hilo al que se espera.

Sintaxis:

```
int pthread_join(pthread_t thread, void **retval);
```

Argumentos:

- pthread_t thread: El identificador del hilo al que se quiere esperar.
- void **retval: Un puntero a un puntero a void. Aquí es donde pthread_join *guardará el valor de retorno* del hilo que ha terminado. Si no te interesa el valor de retorno, puedes pasar NULL.

Valor de retorno:

- pthread_join devuelve 0 si tiene éxito, o un código de error distinto de cero si falla.
-

¿Por qué es importante usar pthread_join?

- *Sincronización:* Asegura que el hilo principal espera a que los hilos que ha creado terminen antes de continuar. Sin pthread_join, el hilo principal podría terminar antes que los hilos hijos, y los hilos hijos podrían quedarse "huérfanos".
- *Liberación de recursos:* Cuando un hilo termina, sus recursos (memoria, etc.) no se liberan automáticamente hasta que otro hilo llama a pthread_join sobre él. Si no llamas a pthread_join, los recursos del hilo se quedarán "colgados" (esto se llama un "hilo zombie", similar a un proceso zombie).
- *Obtener el valor de retorno:* Permite obtener el valor que el hilo devolvió al terminar (usando el argumento `retval`).

3. pthread_exit

La función pthread_exit se usa para *terminar explícitamente* un hilo. Es como un `return` pero específico para hilos.

Sintaxis:

```
void pthread_exit(void *retval);
```

Argumentos:

- void *retval: Un puntero al valor que el hilo quiere devolver al hilo que llame a pthread_join sobre él. Puede ser un puntero a cualquier tipo de dato, o NULL si no quieres devolver nada.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu

Importante!!!: No es necesario llamar explícitamente a pthread_exit al final de la función del hilo. Si la función del hilo llega al final (hace un `return`), implícitamente se llama a pthread_exit.

Importante: Como se ve en el ejemplo, es común que los hilos reserven memoria dinámicamente con malloc y la devuelvan a través de pthread_exit. Es *responsabilidad* del hilo que llama a pthread_join liberar esa memoria.

4. pthread_self

La función pthread_self se usa para obtener el *identificador* del hilo actual. Cada hilo, dentro de un proceso, tiene un ID único (de tipo pthread_t).

Sintaxis:

```
pthread_t pthread_self(void);
```

Valor de retorno:

- pthread_self devuelve el ID del hilo que la llama.

¿Para qué sirve?

- *Depuración:* Para saber qué hilo está ejecutando qué parte del código.
- *Sincronización:* En algunos casos, puede ser útil para implementar mecanismos de sincronización más avanzados.
- *Gestión de recursos:* Para asociar recursos (como estructuras de datos) a hilos específicos.

Compilación (a partir de ahora para los hilos usaremos el -pthread en el gcc):

```
gcc -pthread -o multi_hilo multi_hilo.c
```

3.4 Sincronización y Comunicación

¿Por qué necesitamos Sincronización y Comunicación?

Cuando tienes varios procesos o hilos ejecutándose al mismo tiempo (concurrencia), pueden surgir dos necesidades fundamentales:

1. **Comunicación**: Necesitan intercambiar información. Por ejemplo, un proceso podría calcular un resultado que otro proceso necesita para continuar.
2. **Sincronización**: Necesitan coordinar sus acciones para evitar problemas. Por ejemplo, si dos procesos intentan modificar el mismo archivo al mismo tiempo, el resultado podría ser desastroso.

Sección Extra de Ayuda: Problemas de Concurrencia
Si no se gestionan correctamente la comunicación y la sincronización, pueden ocurrir problemas muy graves y difíciles de depurar:

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.



- *Condiciones de carrera (race conditions):* El resultado del programa depende del orden en que se ejecutan los procesos/hilos, y este orden puede ser impredecible.
- *Interbloqueos (deadlocks):* Dos o más procesos/hilos se bloquean mutuamente, esperando a que el otro libere un recurso que necesita.
- *Inanición (starvation):* Un proceso/hilo nunca consigue acceder a un recurso que necesita, porque otros procesos/hilos se lo "quitan" continuamente.
- *Corrupción de datos:* Si varios procesos/hilos modifican los mismos datos al mismo tiempo sin control, los datos pueden quedar en un estado inconsistente.

Mecanismos de Comunicación

Hay varias formas de comunicar procesos e hilos. Aquí veremos algunas de las más comunes aunque solo estudiaremos un par:

1. Extra: Archivos

Los procesos (y los hilos, ya que comparten el espacio de direcciones) pueden comunicarse a través de archivos. Un proceso escribe datos en un archivo, y otro proceso lee esos datos.

Ventajas:

- Simple de implementar.
- Los datos persisten (se guardan en el disco).

Desventajas:

- Lento (comparado con otros mecanismos). Acceder al disco es mucho más lento que acceder a la memoria.
- Requiere sincronización (si varios procesos escriben en el mismo archivo al mismo tiempo, puede haber problemas).

2. Memoria Compartida (se usa)

La memoria compartida es una región de memoria RAM a la que pueden acceder varios procesos. Es como un "tablón de anuncios" donde los procesos pueden dejar mensajes para otros procesos.

Ventajas:

- *Muy rápido:* La comunicación a través de memoria compartida es la forma más rápida de comunicación entre procesos, ya que no implica copiar datos.

Desventajas:

- *Requiere sincronización:* Es **obligatorio** usar mecanismos de sincronización (como mutexes o semáforos) para evitar condiciones de carrera y asegurar que los procesos no se "pisen" unos a otros al acceder a la memoria compartida.
- *Más complejo de implementar:* Requiere usar funciones especiales del sistema operativo para crear y gestionar la región de memoria compartida.

Los hilos, al pertenecer al mismo proceso, comparten implícitamente la memoria, por lo que este mecanismo no supone un esfuerzo extra.

Procesos (fork()): Los procesos creados con fork, no comparten memoria, por lo que hay que usar llamadas al sistema para usar este tipo de comunicación.



WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3. Extra: Tuberías (Pipes)
Una tubería (pipe) es un canal de comunicación *unidireccional* entre dos procesos. Es como un tubo: un proceso escribe datos en un extremo, y otro proceso lee los datos en el otro extremo. El propio sistema operativo se encarga de almacenar temporalmente los datos que se escriben en la tubería, hasta que el proceso lector los lee (no tienes que usar un archivo para esto).

4. Señales (se usa)
Las señales son un mecanismo de comunicación asíncrono. Son como "avisos" que se envían a un proceso para notificarle que ha ocurrido algún evento. Un proceso puede enviar una señal a otro proceso, o el sistema operativo puede enviar una señal a un proceso.

Ejemplos de señales:

- SIGINT: Se envía cuando el usuario pulsa Ctrl+C en la terminal. Normalmente, causa la terminación del proceso.
- SIGKILL: Se envía para matar a un proceso inmediatamente. El proceso no puede ignorar esta señal.
- SIGTERM: Se envía para pedir a un proceso que termine *amablemente*. El proceso puede ignorar esta señal (a diferencia de SIGKILL).
- SIGCHLD: Se envía al proceso padre cuando un proceso hijo termina.
- SIGUSR1, SIGUSR2: Señales de propósito general que el programador puede usar para lo que quiera.

Funciones para trabajar con señales:

- kill(pid_t pid, int sig): Envía la señal `sig` al proceso con PID `pid`.
- signal(int signum, sighandler_t handler): Establece un manejador de señal. Un manejador de señal es una función que se ejecuta cuando el proceso recibe una señal específica. Esto permite al proceso "interceptar" la señal y realizar alguna acción en lugar de simplemente terminar.
- sigaction: Es una función más avanzada y preferible para manejar señales.

Mecanismos de Sincronización
Cuando varios procesos o hilos comparten recursos (memoria, archivos, etc.), es **obligatorio** *sincronizar* su acceso a esos recursos. La sincronización evita que los procesos/hilos se "pisen" unos a otros y provoquen resultados incorrectos o impredecibles (condiciones de carrera).

Existen varios mecanismos de sincronización. Aquí veremos los más comunes:
1. Mutexes (Exclusión Mutua) (lo usaremos entre hilos)

Un *mutex* (de "mutual exclusion") es un objeto que actúa como un "cerrojo". Solo un hilo (o proceso) puede "adquirir" el cerrojo a la vez. Si otro hilo intenta adquirir el cerrojo mientras

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

ya está adquirido por otro hilo, se bloqueará (se quedará esperando) hasta que el cerrojo se libere.

Funciones (Pthreads):

- pthread_mutex_init: Inicializa un mutex.
- pthread_mutex_lock: Intenta adquirir el cerrojo del mutex. Si el mutex ya está bloqueado por otro hilo, el hilo actual se bloquea hasta que el mutex se libere.
- pthread_mutex_unlock: Libera el cerrojo del mutex.
- pthread_mutex_trylock: Intenta adquirir el cerrojo del mutex, pero*no se bloquea si el mutex ya está bloqueado. Devuelve un valor que indica si pudo adquirir el cerrojo o no.
- pthread_mutex_destroy: Destruye un mutex (libera los recursos asociados)

2. Semáforos (los que tienen nombre los usaremos para procesos)

Un *semáforo* es una variable entera no negativa que se utiliza para controlar el acceso a un recurso compartido por varios procesos o hilos. Es como un contador que indica cuántas "unidades" de un recurso están disponibles.

Operaciones sobre Semáforos:

- wait (o P): Decrementa el valor del semáforo. Si el valor del semáforo es cero, el proceso/hilo se bloquea hasta que el valor sea mayor que cero. Es como "pedir permiso" para usar el recurso.
- signal (o V): Incrementa el valor del semáforo. Si hay procesos/hilos bloqueados esperando al semáforo, uno de ellos se desbloquea. Es como "devolver el permiso".

Tipos de Semáforos:

- *Semáforo binario:* Puede tomar los valores 0 o 1. Es equivalente a un mutex.
- *Semáforo contador:* Puede tomar cualquier valor entero no negativo. Se usa para controlar el acceso a un recurso que tiene varias unidades disponibles.

3. Variables de Condición (práctica 5)

Las variables de condición son otro mecanismo de sincronización que se usa en combinación con los mutexes. Permiten a los hilos *esperar a que se cumpla una condición*, y a otros hilos *señalizar* que la condición se ha cumplido.

Funcionamiento

- Un hilo que necesita esperar a que se cumpla una condición:
    a. Adquiere un mutex.
    b. Comprueba la condición. Si no se cumple:
        - Llama a pthread_cond_wait, pasando la variable de condición y el mutex. Esto hace que el hilo se bloquee y libere el mutex (todo en una sola operación atómica).
        - Cuando otro hilo señalice la variable de condición, el hilo se despertará y volverá a adquirir el mutex automáticamente.
    c. Si la condición se cumple, continúa.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.


Máster en
Inteligencia Artificial y Ciencia de Datos
Impulsa tu perfil en IA y datos
Más info
• APRENDE A TRABAJAR CON DATOS REALES Y MODELOS PREDICTIVOS
• TOMA DECISIONES BASADAS EN ANALÍTICA AVANZADA
• ENFOQUE PRÁCTICO CON CASOS DE NEGOCIO
• SALIDAS EN DATA, IA Y TRANSFORMACIÓN DIGITAL

d. Libera el mutex.

- Un hilo que hace que la condición se cumpla:
  a. Adquiere el mismo mutex.
  b. Modifica las variables que forman parte de la condición.
  c. Llama a pthread_cond_signal (para despertar a un solo hilo que esté esperando) o a pthread_cond_broadcast (para despertar a todos los hilos que estén esperando).
  d. Libera el mutex.

Funciones (Pthreads):

- pthread_cond_init: Inicializa una variable de condición.
- pthread_cond_wait: Bloquea el hilo hasta que la variable de condición sea señalizada. Libera el mutex asociado mientras espera, y lo vuelve a adquirir antes de retornar.
- pthread_cond_signal: Despierta a un hilo (si hay alguno) que esté esperando en la variable de condición.
- pthread_cond_broadcast: Despierta a todos los hilos que estén esperando en la variable de condición.
- pthread_cond_destroy: Destruye una variable de condición.

3.4.1 Concepto de Carrera y Definición de Sección Crítica

¿Qué es una Condición de Carrera?

Una **condición de carrera** (race condition) es un problema que ocurre en programas concurrentes (con varios hilos o procesos) cuando el resultado del programa depende del orden en que se ejecutan los hilos/procesos, y este orden no está controlado. Es decir, el resultado puede ser diferente cada vez que ejecutas el programa, y puede ser incorrecto.

¿Cuándo ocurren las condiciones de carrera?

- Cuando dos o más hilos/procesos acceden a un **recurso compartido** (una variable, un archivo, una estructura de datos, etc.).
- Al menos uno de los hilos/procesos **modifica** el recurso compartido.
- No hay ningún mecanismo de **sincronización** que controle el acceso al recurso.

La Sección Crítica

Una **sección crítica** es un fragmento de código donde un proceso o hilo accede a un recurso compartido (como una variable, un archivo, etc.). El problema es que si varios procesos/hilos entran en la sección crítica al mismo tiempo, pueden ocurrir condiciones de carrera.

La Regla de Oro: Solo un proceso/hilo puede estar dentro de la sección crítica en un momento dado. Esto se llama **exclusión mutua**.

Ejemplo
En el ejemplo anterior, la sección crítica es esta línea:

```
saldo = saldo + cantidad;
```

WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Es crítica porque varios hilos podrían intentar ejecutarla al mismo tiempo, modificando la variable `saldo`.

Para evitar la condición de carrera, necesitamos asegurarnos de que solo un hilo a la vez pueda ejecutar esa línea. Podemos usar un *mutex* (cerrojo de exclusión mutua).

```
pthread_mutex_lock(&mutex);   // Adquirir el cerrojo (entrar al baño)
saldo = saldo + cantidad;       // Sección crítica
pthread_mutex_unlock(&mutex); // Liberar el cerrojo (salir del baño)
```

Ahora, antes de que un hilo pueda entrar en la sección crítica, tiene que "adquirir el cerrojo" (pthread_mutex_lock). Si otro hilo ya tiene el cerrojo, el hilo actual se bloqueará (se quedará esperando) hasta que el cerrojo se libere (pthread_mutex_unlock). De esta forma, se garantiza que solo un hilo a la vez puede modificar la variable saldo.

3.4.2 Exclusión mutua

¿Qué es la Exclusión Mutua?
Cuando tienes varios hilos o procesos que acceden a los mismos datos, necesitas una forma de asegurarte de que solo uno de ellos pueda modificar esos datos a la vez. Esto es lo que se llama *exclusión mutua*.

¿Por qué es necesaria la exclusión mutua? Para evitar las *condiciones de carrera*. Sin exclusión mutua, los datos compartidos podrían corromperse, y el programa podría producir resultados incorrectos o impredecibles.

Mutex: El Cerrojo para la Exclusión Mutua
Un *mutex* (de "mutual exclusion") es un objeto que se utiliza para implementar la exclusión mutua. Funciona como un cerrojo:

- Un hilo/proceso que quiere acceder a un recurso compartido primero tiene que "adquirir" el mutex (cerrar el cerrojo).
- Si el mutex ya está adquirido por otro hilo/proceso, el hilo/proceso actual se bloquea (se queda esperando) hasta que el mutex se libere.
- Cuando un hilo/proceso termina de usar el recurso compartido, "libera" el mutex (abre el cerrojo).
- Solo un hilo/proceso (aunque lo usaremos para hilos) puede tener el mutex adquirido a la vez.

En la biblioteca Pthreads, los mutexes se representan con el tipo de dato pthread_mutex_t.

Funciones para Trabajar con Mutexes (Pthreads)
La biblioteca Pthreads proporciona un conjunto de funciones para crear, destruir y utilizar mutexes.

1. pthread_mutex_init

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu

Propósito: Inicializar un mutex. Antes de poder usar un mutex, debes inicializarlo. Es como preparar el cerrojo antes de poder usarlo.

Sintaxis:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Argumentos:

- pthread_mutex_t *mutex: Un puntero a la variable de tipo pthread_mutex_t que quieres inicializar. Esta variable es el mutex en sí.
- const pthread_mutexattr_t *attr: Un puntero a una estructura de atributos del mutex. Esta estructura permite especificar características avanzadas del mutex (como el tipo de mutex, si es recursivo, etc.). En la mayoría de los casos, puedes pasar NULL para usar los atributos por defecto.

Valor de retorno:

- **Devuelve 0 si la inicialización fue exitosa.**
- Devuelve un código de error distinto de cero si falló.

Inicialización Estática:
Existe una forma más sencilla de inicializar un mutex con los atributos por defecto: usar la macro PTHREAD_MUTEX_INITIALIZER.

```
pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
```

Esto es equivalente a:

```
pthread_mutex_t my_mutex;
pthread_mutex_init(&my_mutex, NULL);
```

2. pthread_mutex_lock

Propósito: Adquirir el cerrojo del mutex. Si el mutex está libre, el hilo actual lo bloquea y continúa. Si el mutex ya está bloqueado por otro hilo, el hilo actual se bloquea (se queda esperando) hasta que el mutex se libere.

Sintaxis:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

Argumentos:

- pthread_mutex_t *mutex: Un puntero al mutex que se quiere bloquear.

Valor de retorno: 0 si tiene éxito, o un código de error si falla.

3. pthread_mutex_unlock

Propósito: Liberar el cerrojo del mutex. Esto permite que otro hilo (si hay alguno esperando) adquiera el mutex y entre en la sección crítica.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH


Máster en
Inteligencia Artificial
& Data Management
Gestiona datos como activo estratégico
Más info
• APRENDE CON PROFESIONALES EN ACTIVO
• METODOLOGÍAS COMO DESIGN THINKING Y CASOS REALES
• PERFIL CLAVE EN ORGANIZACIONES DATA-DRIVEN

Sintaxis:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Argumentos:

- pthread_mutex_t *mutex: Un puntero al mutex que se quiere desbloquear.

Valor de retorno: 0 si tiene éxito, o un código de error si falla.

¡Importante! Solo el hilo que adquirió el mutex puede liberarlo. Intentar liberar un mutex que no ha sido adquirido por el hilo actual, o intentar liberarlo dos veces, es un error y puede causar comportamientos indefinidos.

4. pthread_mutex_destroy

Propósito: Destruir un mutex. Libera los recursos asociados al mutex. Es importante llamar a esta función cuando ya no necesites el mutex.

Sintaxis:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Argumentos:

- pthread_mutex_t *mutex: Un puntero al mutex que se quiere destruir.

Valor de retorno: 0 si tiene éxito, o un código de error si falla.

¡Importante! No intentes destruir un mutex que esté bloqueado por algún hilo. Tampoco intentes usar un mutex después de haberlo destruido.

Pasos para Implementar la Exclusión Mutua con Mutexes (Resumen breve para apuntes)
1) Identificar la sección crítica: Localiza el código que accede al recurso compartido.

Declarar e inicializar un mutex:

```
pthread_mutex_t mi_mutex = PTHREAD_MUTEX_INITIALIZER; // Inicialización estática
// O bien:
pthread_mutex_t mi_mutex;
pthread_mutex_init(&mi_mutex, NULL); // Inicialización dinámica
```

2) Antes de la sección crítica: Adquirir el mutex.

```
pthread_mutex_lock(&mi_mutex);
```

3) Dentro de la sección crítica: Acceder/modificar el recurso compartido.
Después de la sección crítica: Liberar el mutex.

```
pthread_mutex_unlock(&mi_mutex);
```

4) Cuando ya no se necesite el mutex: Destruirlo.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
pthread_mutex_destroy(&mi_mutex);
```

Ejemplo Completo de cómo se ve un programa con mutex

```
#include pthread.h
#include stdio.h
#include stdlib.h

#define NUM_HILOS 2
#define INCREMENTOS 1000000

int contador = 0; // Variable global compartida: ¡SECCIÓN CRÍTICA!
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // Mutex para proteger el contador

void *incrementar(void *arg) {
    int i;
    for (i = 0; i < INCREMENTOS; i++) {
        // --- Adquirir el mutex (entrar en la sección crítica) ---
        pthread_mutex_lock(&mutex);

        // --- SECCIÓN CRÍTICA ---
        contador++;
        // --- SECCIÓN CRÍTICA ---

        // --- Liberar el mutex (salir de la sección crítica) ---
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t hilos[NUM_HILOS];
    int i;

    // Crear los hilos
    for (i = 0; i < NUM_HILOS; i++) {
        if (pthread_create(&hilos[i], NULL, incrementar, NULL) != 0) {
            perror("pthread_create failed");
            exit(EXIT_FAILURE);
        }
    }

    // Esperar a que los hilos terminen
    for (i = 0; i < NUM_HILOS; i++) {
        pthread_join(hilos[i], NULL);
    }

    // Destruir el mutex (ya no lo necesitamos)
    pthread_mutex_destroy(&mutex);

    printf("Contador final: %d (debería ser %d)\n", contador, NUM_HILOS * INCREMENTOS);
    return 0;
}
```

Compilación

```
gcc -pthread -o mutex_ejemplo mutex_ejemplo.c
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

Explicación:

- int contador = 0;: Variable global compartida. Esta es la variable que vamos a proteger con el mutex.
- pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;: Declaración e inicialización del mutex. Usamos la macro PTHREAD_MUTEX_INITIALIZER para inicializarlo con los valores por defecto.
- void *incrementar(void *arg): Función que incrementa el contador un millón de veces.
- pthread_mutex_lock(&mutex);: Adquiere el cerrojo del mutex antes de acceder a la variable compartida.
- contador++;: La sección crítica. Aquí se modifica la variable compartida.
- pthread_mutex_unlock(&mutex);: Libera el cerrojo del mutex inmediatamente después de salir de la sección crítica.
- En main:
    - Se crean dos hilos que ejecutan la función incrementar.
    - Se espera a que los dos hilos terminen (con pthread_join).
    - Se destruye el mutex (con pthread_mutex_destroy).
    - Se imprime el valor final del contador.

Si ejecutas este programa, verás que el valor final del contador es siempre 2000000 (2 hilos * 1000000 incrementos/hilo). Si quitas las llamadas a pthread_mutex_lock y pthread_mutex_unlock, obtendrás un resultado incorrecto (y diferente cada vez que ejecutes el programa) debido a la condición de carrera.

Recordad todo esto:

- Identificar la sección crítica: En este caso la variable global contador.
- Declarar e inicializar el mutex: Se declara la variable mutex de tipo pthread_mutex_t y se inicializa.
- Antes de la sección crítica: Se adquiere el cerrojo con la función pthread_mutex_lock().
- Sección crítica: Se modifica la variable.
- Después de la sección crítica: Se libera el mutex con la función pthread_mutex_unlock().

3.4.3 Semáforos, cerrojos y variables condicionales

Introducción

Ya hemos visto cómo los mutexes (cerrojos) nos permiten proteger secciones críticas y evitar condiciones de carrera. Pero los mutexes son solo uno de los mecanismos de sincronización que existen. Ahora veremos otros dos: las *variables de condición* y los *semáforos*.

Variables de Condición

Una variable de condición es un mecanismo de sincronización que permite a los hilos *esperar* hasta que se cumpla una determinada condición, y a otros hilos *señalizar* que esa condición se ha cumplido.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu






Analogía: Imagina una sala de espera en un médico. Los pacientes (hilos) llegan y se sientan a esperar. Cuando el médico (otro hilo) está libre, llama al siguiente paciente (señaliza la variable de condición). Los pacientes no están todo el rato preguntando "¿Ya? ¿Ya? ¿Ya?". Simplemente esperan, y el médico les avisa cuando es su turno.

Importante: Las variables de condición siempre se usan en combinación con un *mutex*. El mutex se usa para proteger el acceso a la variable que representa la condición (para evitar condiciones de carrera al comprobar o modificar la condición).

Funciones (Pthreads):

- pthread_cond_init: Inicializa una variable de condición.
- pthread_cond_wait:
  - Bloquea el hilo actual hasta que la variable de condición sea señalizada.
  - *Libera* el mutex asociado mientras espera.
  - Cuando el hilo se despierta (porque otro hilo ha señalizado la variable de condición), vuelve a adquirir el mutex antes de retornar. Todo esto ocurre de forma atómica (como una sola operación).
- pthread_cond_signal: Despierta a un hilo (si hay alguno) que esté esperando en la variable de condición. Si hay varios hilos esperando, solo uno se despierta (no se puede predecir cuál).
- pthread_cond_broadcast: Despierta a todos los hilos que estén esperando en la variable de condición.
- pthread_cond_destroy: Destruye una variable de condición.

Sintaxis:

```
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Argumentos:

- pthread_cond_t *cond: Puntero a la variable de condición.
- pthread_mutex_t *mutex: Puntero al mutex asociado a la variable de condición.
- pthread_condattr_t *attr: Puntero a una estructura de atributos (o NULL para los atributos por defecto). No lo usaremos en esta práctica.

Semáforos

Un *semáforo* es una variable entera no negativa que se utiliza para controlar el acceso a un recurso compartido por varios procesos o hilos. Es como un contador que indica cuántas "unidades" de un recurso están disponibles.

Operaciones sobre semáforos

- wait (o P): *Decrementa* el valor del semáforo. Si el valor del semáforo es cero, el proceso/hilo se *bloquea* hasta que el valor sea mayor que cero. Es como "pedir permiso" para usar el recurso.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- signal (o V): *Incrementa* el valor del semáforo. Si hay procesos/hilos bloqueados esperando al semáforo, uno de ellos se desbloquea. Es como "devolver el permiso".

Tipos de semáforos

- *Semáforo binario*: Puede tomar los valores 0 o 1. Es equivalente a un mutex.
- *Semáforo contador*: Puede tomar cualquier valor entero no negativo. Se usa para controlar el acceso a un recurso que tiene varias unidades disponibles.

En la librería pthreads, se pueden implementar semáforos, pero no están incluidos dentro de esta librería, por lo que usaremos la librería semaphore.h

Ejemplo (semáforo POSIX):

```
#include stdio.h
#include pthread.h
#include stdlib.h
#include semaphore.h //Para los semáforos

#define NUM_HILOS 5
#define NUM_RECURSOS 2 // Recursos disponibles

sem_t semaforo; // Declaración del semáforo

void *hilo(void *arg) {
    int id = *(int *)arg;
    printf("Hilo %d: Intentando adquirir semáforo...\n", id);
    sem_wait(&semaforo); // Esperar (P)
    printf("Hilo %d: Semáforo adquirido. Usando recurso...\n", id);
    sleep(1); // Simular uso del recurso
printf("Hilo %d: Liberando semáforo.\n", id);
    sem_post(&semaforo); // Señalar (V)
    pthread_exit(NULL);
}

int main() {
    pthread_t hilos[NUM_HILOS];
    int ids[NUM_HILOS];
    int i;

    // Inicializar el semáforo con 2 unidades
    if (sem_init(&semaforo, 0, NUM_RECURSOS) != 0) {
        perror("sem_init failed");
        exit(1);
    }

    // Crear los hilos
    for (i = 0; i < NUM_HILOS; i++) {
        ids[i] = i;
        if (pthread_create(&hilos[i], NULL, hilo, &ids[i]) != 0) {
            perror("pthread_create failed");
            exit(1);
        }
    }

    // Esperar a que todos los hilos terminen
    for (i = 0; i < NUM_HILOS; i++) {
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compartir coche en la complu no es nuevo. Recibir dinero por ello, sí. - Tribbu

```
        pthread_join(hilos[i], NULL);
    }

    // Destruir el semáforo
    sem_destroy(&semaforo);

    return 0;
}
```

Explicación:

1. #include : Incluimos el archivo de cabecera necesario para usar semáforos POSIX.
2. sem_t semaforo;: Declaramos una variable de tipo sem_t. Este es el tipo de dato para los semáforos POSIX.
3. sem_init(&semaforo, 0, NUM_RECURSOS);: Inicializamos el semáforo.
    - &semaforo: La dirección del semáforo que queremos inicializar.
    - 0: Indica que el semáforo se comparte entre hilos del mismo proceso (si fuera distinto de cero, se compartiría entre procesos).
    - NUM_RECURSOS: El valor inicial del semáforo (el número de "recursos" disponibles). En este caso, 2.
4. sem_wait(&semaforo);: El hilo intenta "adquirir" el semáforo (operación P o wait).
    - Si el valor del semáforo es mayor que 0, el valor se decrementa en 1, y el hilo continúa.
    - Si el valor del semáforo es 0, el hilo se *bloquea* hasta que otro hilo incremente el valor del semáforo.
5. sem_post(&semaforo);: El hilo "libera" el semáforo (operación V o signal). Incrementa el valor del semáforo en 1. Si hay hilos bloqueados esperando al semáforo, uno de ellos se desbloquea.
6. sem_destroy(&semaforo);: Destruye el semáforo. Es importante destruir los semáforos cuando ya no se necesitan.

En este ejemplo, hay 5 hilos y el semáforo se inicializa con un valor de 2. Esto significa que, como máximo, 2 hilos podrán estar en la sección crítica al mismo tiempo. El resto tendrá que esperar a que el semáforo tenga un valor mayor que cero (a que un hilo ejecute sem_post). Este tipo de semáforo que puede tomar valores mayores que 1 se llama semáforo contador. Si solo pudiese tomar los valores 0 y 1, sería un semáforo binario (equivalente a un mutex).

Semáforos con Nombre vs. Semáforos Anónimos

En el ejemplo anterior, hemos usado un *semáforo anónimo*. Los semáforos anónimos se almacenan en la memoria del proceso y son útiles para sincronizar hilos dentro del mismo proceso.

Pero también existen los *semáforos con nombre*. Los semáforos con nombre se identifican por un nombre (una cadena de caracteres) y pueden ser compartidos entre procesos diferentes. Son una forma de comunicación entre procesos (IPC).

Funciones para Semáforos con Nombre:

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

SHÔKO MADRID
EL EXAMEN SE APRUEBA ESTUDIANDO,
LA CARRERA SE VIVE SALIENDO.
Dale el último empujón a los apuntes y vente.
Pilla tu entrada
Nosotros ponemos la música y las copas para cuando termines el tema. Remata la faena y asegúrate la entrada ya, que vuelan.
Domingo
Sábado
Viernes
Jueves
Lunes

- sem_open: Crea o abre un semáforo con nombre.
- sem_close: Cierra un semáforo con nombre.
- sem_unlink: Elimina un semáforo con nombre.
- sem_wait, sem_post: Funcionan igual que con los semáforos anónimos.

Importante: Los semáforos con nombre, al ser un mecanismo de comunicación entre procesos, requieren que los procesos que los usan compartan memoria (de forma explícita). Los semáforos anónimos se usan para sincronizar hilos de un mismo proceso (que ya comparten memoria implícitamente). Los semáforos con nombre, al ser un recurso global del sistema, se usan para sincronizar procesos diferentes, que por defecto no comparten memoria.

Memoria Compartida entre Procesos

La memoria compartida es un mecanismo de comunicación entre procesos (IPC) que permite a varios procesos acceder a la misma región de memoria. A diferencia de las tuberías o colas de mensajes, donde los datos se copian de un proceso a otro, en la memoria compartida los datos están realmente en el mismo lugar físico, y todos los procesos que tienen acceso a esa región pueden ver los cambios realizados por otros procesos inmediatamente. Esto la convierte en una forma muy eficiente de comunicación, especialmente para grandes cantidades de datos.

¡Importante! La memoria compartida es un mecanismo de comunicación entre procesos. Los hilos, por definición, ya comparten la memoria del proceso al que pertenecen, por lo que no necesitan mecanismos especiales de memoria compartida.

Funciones para la Memoria Compartida (POSIX)

Vamos a utilizar las funciones de memoria compartida POSIX (Portable Operating System Interface), que son las más comunes en sistemas tipo Unix (Linux, macOS, etc.). Las funciones principales son:

- shm_open: Crea o abre un objeto de memoria compartida. Es como "crear el tablón de anuncios".
- ftruncate: Establece el tamaño del objeto de memoria compartida. Es como "definir el tamaño del tablón".
- mmap: "Mapea" el objeto de memoria compartida en el espacio de direcciones del proceso. Es decir, hace que el objeto de memoria compartida sea accesible como si fuera un array normal en el programa. Es como "dar acceso al tablón" a un empleado.
- munmap: "Desmapea" el objeto de memoria compartida. Es decir, deja de hacer que sea accesible desde el proceso. Es como "quitar el acceso al tablón".
- shm_unlink: Elimina el objeto de memoria compartida (cuando ya no se necesita). Es como "quitar el tablón".

Importante: Para usar estas funciones, se deben incluir estas cabeceras:

```
#include sys/mman. // Para mmap, munmap
#include sys/stat.h // Para los modos (O_RDONLY, etc.)
```

WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
#include fcntl.h // Para O_CREAT, O_RDWR, etc.
#include unistd.h // Para ftruncate
#include sys/shm.h //Para shm_open y shm_unlink
```

Creación y Uso de Memoria Compartida (Paso a Paso)

Crear/Abrir el Objeto de Memoria Compartida (shm_open):

```
int shm_open(const char *name, int oflag, mode_t mode);
```

- name: Un nombre para el objeto de memoria compartida. Debe empezar con una barra (/). Es como el nombre del tablón de anuncios.
- oflag: Flags que indican cómo se debe abrir el objeto. Los más importantes son:
    - O_CREAT: Crea el objeto si no existe.
    - O_RDWR: Abre el objeto para lectura y escritura.
    - O_RDONLY: Abre el objeto para solo lectura.
    - O_EXCL: Si se combina con O_CREAT, falla si el objeto ya existe.
- mode: Permisos del objeto (si se crea). Similar a los permisos de los archivos (ej: 0666).
- Valor de retorno: Devuelve un *descriptor de archivo* (un entero no negativo) si tiene éxito, o -1 si falla.

Establecer el Tamaño (ftruncate):

```
int ftruncate(int fd, off_t length);
```

- fd: El descriptor de archivo devuelto por shm_open.
- length: El tamaño (en bytes) que quieres que tenga el objeto de memoria compartida.
- Valor de retorno: 0 si tiene éxito, o -1 si falla.

Es obligatorio establecer el tamaño del objeto de memoria compartida antes de poder usarlo.

Mapear la Memoria (mmap):

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- addr: Normalmente se pasa NULL, para que el sistema operativo elija la dirección donde mapear la memoria.
- length: El tamaño (en bytes) de la región de memoria que quieres mapear (debe ser igual o menor que el tamaño del objeto de memoria compartida).
- prot: Protección de la memoria. Puede ser una combinación de:
    - PROT_READ: La memoria se puede leer.
    - PROT_WRITE: La memoria se puede escribir.
    - PROT_EXEC: La memoria se puede ejecutar (normalmente no se usa para memoria compartida).
    - PROT_NONE: La memoria no se puede acceder.
- flags: Flags que controlan el comportamiento del mapeo. Los más importantes son:

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

- MAP_SHARED: Las modificaciones en la memoria se comparten con otros procesos que mapeen el mismo objeto. Este es el que debes usar para memoria compartida.
- MAP_PRIVATE: Las modificaciones son privadas del proceso.
    - fd: El descriptor de archivo devuelto por shm_open.
    - offset: Normalmente 0 para mapear el objeto completo.
    - Valor de retorno: Devuelve un puntero a la región de memoria mapeada, o MAP_FAILED ((void*)-1) si falla.

Una vez que has mapeado el objeto de memoria compartida, puedes acceder a él como si fuera un array normal en tu programa.

Desmapear la Memoria (munmap):

```
int munmap(void *addr, size_t length);
```

- addr: La dirección de inicio de la región de memoria que quieres desmapear (la que te devolvió mmap).
- length: El tamaño de la región de memoria.
- Valor de retorno: 0 si éxito, -1 si falla.

Cuando un proceso termina de usar la memoria compartida, debe desmapearla.

Eliminar el Objeto de Memoria Compartida (shm_unlink):

```
int shm_unlink(const char *name);
```

name: El nombre del objeto de memoria compartida (el mismo que usaste en shm_open).

- Valor de retorno: 0 si éxito, -1 si falla.

Cuando ya no se necesita el objeto de memoria compartida, se debe eliminar. Esto no libera la memoria inmediatamente; la memoria se libera cuando todos los procesos que tienen mapeado el objeto lo han desmapeado (con munmap) o han terminado. Es como quitar el tablón de anuncios de la pared.

3.4.4 Soporte Hardware para Primitivas de Sincronización

La sincronización de hilos y procesos sirve para evitar condiciones de carrera y garantizar la exclusión mutua en sistemas concurrentes. El hardware moderno ofrece soporte especializado para implementar primitivas de sincronización de manera eficiente, mejorando el rendimiento y la escalabilidad de las aplicaciones.

Primitivas de Sincronización y su Implementación con Hardware

Las primitivas de sincronización, como cerrojos (mutexes), variables de condición y semáforos, dependen de operaciones atómicas para su correcta implementación. Estas operaciones atómicas son garantizadas por el hardware.

Cerrojos (Mutexes): Aseguran que solo un hilo acceda a una sección crítica a la vez. Se implementan utilizando instrucciones atómicas a nivel de hardware.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.

knoweats!
Estudiar, dormir y comer bien
Elige las tres. Con Knoweats lo tienes fácil: comida casera lista para que te centres en lo demás.
Comer bien



20€ DE DESCUENTO
con el código 20ESTUDIO

Instrucciones Atómicas Clave:

Test-and-Set (TAS): Lee una ubicación de memoria y, en la misma operación indivisible, escribe un valor (generalmente 1, indicando "bloqueado"). El valor anterior leído es retornado. Si el valor retornado es 0 (desbloqueado), el hilo ha adquirido el cerrojo; si es 1, el cerrojo ya estaba tomado.

Compare-and-Swap (CAS): Compara el valor actual de una ubicación de memoria con un valor esperado. Si son iguales, escribe un nuevo valor (generalmente indicando bloqueo) en la ubicación de memoria. Esta operación es atómica. Retorna un valor booleano indicando si el intercambio tuvo éxito (el valor era el esperado) o no. Es más flexible que TAS.

Load-Link/Store-Conditional (LL/SC): (presente en arquitecturas como ARM y MIPS). Un par de instrucciones:

- *Load-Link (LL)*: Lee el valor de una dirección de memoria.
- *Store-Conditional (SC)*: Intenta escribir un nuevo valor en la misma dirección. La escritura *solo* tiene éxito si ningún otro procesador/núcleo ha modificado la memoria desde la instrucción LL. SC retorna un valor indicando éxito o fracaso. Permite implementar operaciones atómicas más complejas que TAS o CAS.

Fetch-and-Add (FAA): Incrementa atómicamente el valor de una ubicación de memoria y devuelve el valor *anterior*. Útil para implementar contadores atómicos, y a partir de ahí, otras primitivas.

Ventajas del Hardware: Las instrucciones atómicas a nivel de hardware garantizan que la adquisición y liberación de un cerrojo sean operaciones indivisibles. Esto evita condiciones de carrera y asegura la exclusión mutua. Sin soporte hardware, implementar exclusión mutua sería extremadamente complejo, ineficiente y propenso a errores.

Variables de Condición: Permiten que los hilos se bloqueen esperando que una condición específica se cumpla y sean notificados (despertados) por otros hilos cuando la condición cambia. Se usan siempre en conjunto con un cerrojo (mutex). El mutex protege la condición y asegura que las operaciones sobre la variable de condición sean atómicas. Aunque las variables de condición en sí mismas no suelen tener soporte directo a nivel de hardware específico (como las instrucciones atómicas), se benefician indirectamente del soporte hardware para mutexes.

Semáforos: Los semáforos son una generalización de los mutexes. Permiten controlar el acceso a un recurso compartido por un número limitado de hilos (no solo uno, como el mutex). Al igual que las variables de condición, se implementan usando mutexes para proteger su estado interno (el contador del semáforo).

Monitores (Concepto de Alto Nivel)
Un monitor es una construcción de sincronización de alto nivel que encapsula datos compartidos y las operaciones que acceden a ellos, proporcionando exclusión mutua y mecanismos para la sincronización basada en condiciones. Un monitor no es una primitiva proporcionada directamente por el hardware, sino una abstracción construida usando las primitivas de bajo nivel (mutexes y variables de condición, que a su vez se apoyan en el hardware).

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Componentes Clave de un Monitor:

- **Datos Compartidos**: Las variables o estructuras de datos que el monitor protege.
- **Procedimientos (Métodos)**: Las únicas operaciones que pueden acceder a los datos compartidos.
- **Mutex Implícito**: Asegura que solo un hilo pueda estar ejecutando un procedimiento del monitor a la vez (exclusión mutua).
- **Variables de Condición (Opcionales)**: Permiten a los hilos dentro del monitor esperar a que se cumplan ciertas condiciones y ser notificados cuando cambian.

Ventajas del Soporte Hardware

- **Eficiencia**: Las instrucciones atómicas a nivel de hardware son mucho más rápidas que las soluciones implementadas puramente en software.
- **Escalabilidad**: En sistemas multiprocesador o multinúcleo, el soporte hardware es esencial para la escalabilidad. Las instrucciones atómicas permiten una sincronización eficiente entre múltiples núcleos.
- **Simplicidad** (relativa): Aunque la programación concurrente sigue siendo compleja, el soporte hardware simplifica la implementación de las primitivas de sincronización en comparación con soluciones puramente en software.
- **Corrección**: Reduce la probabilidad de errores (como condiciones de carrera) que son difíciles de depurar en programas concurrentes.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Haz TRIBBU hasta la complu, gana tiempo y dinero.